# فصل اول

**فصل اول: محاسبات موازی بر روی سیستم‌های ناهمگون..... ۲**

۱ موازی‌سازی..... ۲

۲ معماری کامپیوتر..... ۴

۳ محاسبات ناهمگون یا **HETEROGENEOUS**..... ۷

۴ معماری **heterogeneous** یا نامتجانس (ناهمگون)..... ۸

۵ قابلیت محاسباتی یا **compute capabilitiy**..... ۱۰

۶ الگوی محاسباتی ناهمگون..... ۱۱

۷ **CUDA** : یک پلتفرم برای محاسبات ناهمگون..... ۱۴

۸ آیا برنامه‌نویسی با **CUDA** مشکل است؟..... ۱۹

۹ محیط توسعه‌ی **CUDA**..... ۲۰

**فصل دوم : مدل برنامه‌نویسی CUDA..... ۲۱**

۱ معرفی مدل برنامه‌نویسی **CUDA**..... ۲۱

۲ ساختار برنامه‌نویسی **CUDA**..... ۲۲

۳ مدیریت حافظه..... ۲۴

۳.۱ سلسله مراتب حافظه **GPU** (خلاصه)..... ۲۶

۳.۲ تفاوت فضای حافظه **host** و **device**..... ۲۹

۳.۳ چرا در زبان **c** از **pointer to pointer** استفاده می‌شود..... ۲۹

۴ مدیریت تردها..... ۳۰

۴.۱ ابعاد **GRID** و **BLOCK**..... ۳۴

۴.۲ دسترسی به متغیرهای **GRID/BLOCK** از سمت **HOST** و **DEVICE**..... ۳۷

۴.۳ سلسله مراتب ترد..... ۳۸

۵ راه‌اندازی یک کرنل **CUDA**..... ۳۹

۵.۱ رفتار آسنکرون ..... ۴۰
۶ نوشتن کرنل ..... ۴۱
۶.۱ توابع کرنل، توابع دارای محدودیت هستند ..... ۴۱
۷ صحت سنجی کرنل ..... ۴۲
۷.۱ تصدیق کد کرنل ..... ۴۳
۸ رسیدگی به خطاها ..... ۴۳
۹ کامپایل و اجرا ..... ۴۴

# فصل اول: محاسبات موازی بر روی سیستم‌های ناهمگون

## ۱ موازی‌سازی

امروزه، موازی سازی در همه‌جا وجود دارد، و برنامه‌نویسی موازی به عنوان یک مبحث جدایی ناپذیر در دنیای برنامه‌نویسان به شمار میآید. موازی‌سازی در چندین سطوح، مرتبط با معماری طراحی می باشد. به صورت کلی دو نوع موازی‌سازی وجود دارد:

- موازی‌سازی **Task**
- موازی‌سازی **Data**

موازی‌سازی **Task** وقتی کاربرد پیدا می‌کند که چندین **task** و یا چندین تابع وجود دارد که می‌توانند به صورت مستقل از همدیگر کار کنند. در واقع موازی‌سازی در مورد **Task** تلاش می‌کند که توابع را بر روی هسته‌های مختلف تقسیم کند.

موازی‌سازی **Data** وقتی کاربرد پیدا می‌کند که داده‌های زیادی وجود دارد که امکان پردازش همزمان بر روی آن‌ها وجود دارد. در واقع موازی‌سازی **Data** بر روی تقسیم داده‌ها بر روی چندین هسته تمرکز دارد.

برنامه‌نویسی **CUDA** برای مسایلی که در آن‌ها امکان محاسبات موازی بر روی داده‌ها وجود دارد، بسیار مناسب به شمار میآید. عمده تمرکز این کتاب نحوه حل برنامه نویسی موازی بر روی داده ها با استفاده از برنامه‌نویسی **CUDA** می‌باشد. بسیاری از برنامه‌هایی که برای پردازش داده‌های بزرگ هستند می‌توانند از مدل **data-parallel** برای افزایش سرعت محاسبات خود بهره ببرند. پردازش **data-parallel** داده‌ها را بر روی تردهای موازی نگاشت می‌کند.

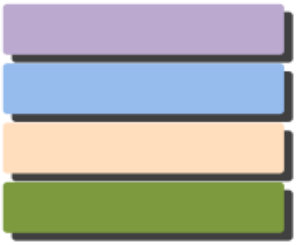
اولین گام در طراحی برنامه **data-parallel** این است که داده‌ها را بر روی تردها پخش کنیم، که هر ترد وظیفه‌ی کار بر روی قسمتی از داده را بر عهده میگیرد. در حالت کلی، دو رویکرد برای تقسیم دیتا وجود دارد: تقسیم **block** ی و تقسیم **cyclic** . در تقسیم **block** ی، بسیاری از المان‌های متوالی از داده، به صورت دسته‌ای در کنار هم هستند. هر دسته بر روی یک ترد انتساب داده شده، و تردها در هر لحظه فقط یک دسته از این داده‌ها را پردازش می‌کنند. در تقسیم **cyclic** ،میزان داده کمتری در دسته‌ها قرار میگیرد، پس طبیعتا تعداد دسته‌ها بیشتر از روش **block** ی خواهد بود. تردهای همسایه، دسته‌های همسایگی را نیز دریافت می‌کنند و هر ترد می‌تواند بیش از یک دسته را پردازش کند. انتخاب یک دسته جدید برای پردازش بر روی ترد، موجب پریدن هر چند دسته که در ترد است می‌شود. تصویر زیر دو

مثال از نحوه‌ی تقسیم دیتای یک بعدی را نمایش میدهد. در تقسیم **block** ی، هر ترد تنها یک قسمت از داده‌ها را برای پردازش برمیدارد، و در تقسیم **cyclic** ، هر ترد بیشتر از یک دسته را برای پردازش برمیدارد.( هر رنگ مرتبط با یک ترد است، پس در این مورد چهار ترد وجود دارد).

*Block partition: each thread takes one data block*

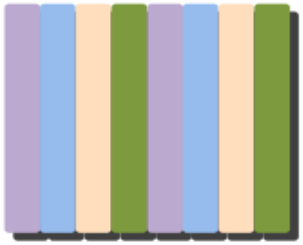
*Cyclic partition: each thread takes two data blocks*

در تصویر زیر سه مثال ساده از دیتای دو بعدی را نشان میدهد: تقسیم **block**ی در راستای محور **y**، تقسیم **block** ای در راستای هر دو محور، و تقسیم **cyclic** در راستای محور **x** . بقیه‌ی حالت‌های ممکن به عنوان تمرین به عهده خودتان گذاشته می‌شود.



*Block partition on one dimension*

*Block partition on both dimensions*



*Cyclic partition on one dimension*

معمولا، داده به صورت یک بعدی ذخیره می‌شود، حتی اگر یک دید چند بعدی از داده داشته باشیم، همچنان بر روی حافظه به صورت یک بعدی نگاشت می‌شود. مشخص کردن این که چطور داده بر روی تردها تقسیم شود معمولا وابسته به این است که به صورت فیزیکی چطور ذخیره شده است، همچنین این که ترتیب تردها به چه صورت می‌باشد. این که چطور تردها را مدیریت می‌کند تاثیر زیادی بر روی عملکرد کلی برنامه خواهد داشت.

عملکرد برنامه معمولا حساس به **block size** هست. مشخص کردن بهترین میزان تقسیم در روش **block** ی و در روش **cyclic** به معماری کامپیوتر وابسته است. در این مورد در ادامه بیشتر یاد خواهیم گرفت.

## ۲ معماری کامپیوتر

چندین روش برای طبقه‌بندی کردن معماری کامپیوترها وجود دارد. یکی از این روش‌ها که خیلی استفاده می‌شود طبقه‌بند فلین است، که معماری کامپیوتر را به چهار دسته مختلف تقسیم می‌کند، این طبقه بندی بر اساس نحوه‌ی جریان یافتن **instructions** و **data** بر روی هسته‌های سیستم عامل هست که شامل موارد زیر می‌شود:

- Single Instruction Single Data (SISD)
- Single Instruction Multiple Data (SIMD)
- Multiple Instruction Single Data (MISD)
- Multiple Instruction Multiple Data (MIMD)

**معماری یک دستورالعمل یک داده** اشاره به کامپیوترهای قدیمی دارد: یک معماری سریال. که تنها یک هسته در چنین سیستم عاملی وجود دارد. و در هر لحظه تنها یک جریان دستورالعمل قادر به اجراست، و عملیات‌ها بر روی یک جریان داده اعمال می‌شوند.

**یک دستورالعمل چند داده اشاره** به نوعی از معماری موازی دارد. در این معماری چندین هسته وجود دارد. همه‌ی هسته‌ها یک جریان دستورالعمل را در هر لحظه اجرا می‌کنند، و هر کدام بر روی جریان داده‌های مختلف اجرا می‌شوند. کامپیوترهای برداری معمولا از نوع **SIMD** هستند، و بیشتر کامپیوترهای مدرن یک معماری **SIMD** را به کار میگیرند. شاید بزرگترین مزیت **SIMD** این است که، در لحظه نوشتن کد بر روی **CPU** ، برنامه‌نویس می‌تواند به صورت سریال فکر کند ولی به پرفرمنس یک عملیات موازی دست پیدا کند چرا که کامپایلر جزییات پیاده‌سازی را برعهده خواهد گرفت.

**چندین دستورالعمل یک داده** به یک معماری غیر رایج اشاره دارد، که هر هسته بر روی یک جریان داده‌ی یکسان با چندین جریان دستورالعملی متفاوت عملیاتی را انجام میدهد.

**چندین دستورالعمل چندین داده** به معماری رایج موازی سازی اشاره دارد که در آن چندین هسته بر روی چندین جریان داده عملیاتی را انجام میدهد، و هر کدام به صورت مستقل دستورالعمل‌هایی را اجرا می‌کنند. بسیاری از معماری‌های **MIMD** یک کامپوننت زیرین **SIMD** را نیز در خود دارند.

در سطح معماری، پیشرفت‌های زیادی به منظور رسیدن به موفقیت‌های زیر صورت پذیرفته است:

- کاهش تاخیر
- افزایش پهنای باند
- افزایش توان عملیاتی

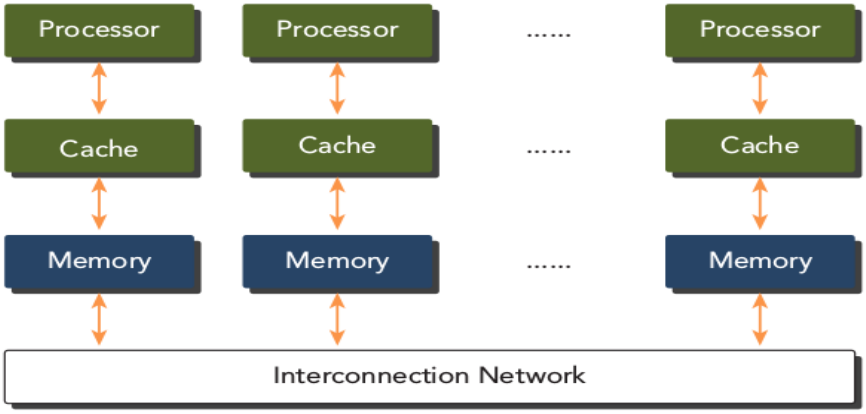
**تاخیر یا latency** به میزان زمانی اطلاق می‌شود که یک عملیات شروع و پایان می‌پذیرد، و معمولا در واحد میکروثانیه بیان می‌شود. **پهنای باند** مقدار داده‌ای است که می‌تواند در یک واحد زمانی پردازش شود، معمولا پهنای باند را تحت **megabytes/sec** و یا **gigabytes/sec** بیان می‌کنند. **توان عملیاتی** میزان دستوراتی است که می‌توان در یک واحد زمانی انجام داد، که معمولا تحت عنوان **gflops** بیان می‌شود(**giga float operation per second**) ، مخصوصا در فیلدهای علمی که محاسبات سنگین بر روی محاسبات ممیز شناور انجام می‌شود. تاخیر زمان سپری شده برای تکمیل شدن یک عملیات را نشان میدهد، در حالی که توان عملیاتی تعداد دستوراتی است که می‌توان در یک واحد زمانی انجام داد را نمایش میدهد.

معماری کامپیوتر همچنین می‌تواند بر اساس سازماندهی حافظه تقسیم شود، بر این مبنا کامپیوترها به دو دسته تقسیم می‌شوند.

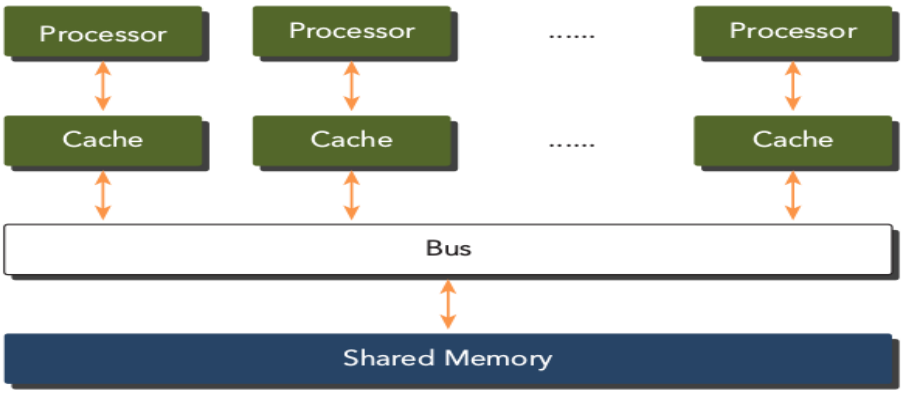
➤ **Multi-node with distributed memory**(چند نودی با حافظه توزیع شده)

➤ **Multiprocessor with shared memory**( چندپردازشگری با حافظه اشتراکی)

در یک سیستم چند نودی، موتورهای محاسباتی با ابعاد بزرگ ساخته شده که با شبکه به هم متصل هستند. هر پردازشگر حافظه محلی خود را دارد، و پردازشگرها می‌توانند محتوای حافظه محلی خود را بر روی شبکه به اشتراک بگذارند. شکل زیر یک سیستم چند نودی با حافظه توزیع شده را نمایش میدهد. چنین سیستم‌هایی را معمولا کلاستر می‌نامند.




معماری چند پردازنده‌ای (چند هسته‌ای) معمولا از رنج دو پردازنده‌ای(**dual processor**) تا هزاران یا صدها پردازنده می‌باشد. این پردازنده‌ها به صورت فیزیکی به یک حافظه مشترک متصل هستند، که در شکل بعدی مشخص می‌باشد، یا این که با یک لینک با تاخیر پایین(مثل **PCI-Express** یا **PCIe**) به هم متصل هستند.




اصطلاح **many-core** معمولا برای توصیف معماری چندهسته‌ای مخصوصا با تعداد هسته‌های بالا (ده ها یا صدها هسته) استفاده می‌شود. اخیرا، معماری کامپیوترها در حال انتقال از **multi-core** به **many-core** می‌باشد.

**GPU** ها یک معماری **many-core** دارند، و به صورت **مجازی** همه‌ی انواع موازی‌سازی که قبلا بیان شد را دارند: **SIMD**، **MIMD**، **multi-threading** و موازی سازی در سطح دستورات. **NVIDIA** از عبارت **single Instruction,Multiple thread(SIMT)** برای چنین معماری استفاده می‌کند.

**CPU** و **GPU** تاریخچه‌ی متفاوتی دارند. در واقع، **GPU** ها در گذشته به عنوان یک شتاب‌دهنده استفاده میشد. ولی اخیرا **GPU** ها خیلی قدرتمند، همه‌کاره، کاملا برنامه‌پذیر، قابلیت موازی‌سازی **task** و **data** را پیدا کرده‌اند و برای مسایلی که درگیر محاسبات موازی هستند بسیار ایده‌آل هستند.

### تفاوت GPU و CPU در این دیدگاه

اگر چه ممکن است هم برای **CPU** و هم برای **GPU** از اصطلاح **many-core** استفاده شود ولی **GPU** از لحاظ معماری کاملا متفاوت از یک هسته **CPU** است.

یک هسته‌ی **CPU**، نسبتا سنگین وزن بوده، و برای محاسبات منطقی خیلی پیچیده طراحی شده است، هدف در این طراحی بهینه کردن محاسبات سریال و پشت سر هم می‌باشد.

یک هسته‌ی **GPU** ، نسبتا سبک وزن، و برای کارهای **data-parallel** با منطق ساده‌تر بهینه شده است، و تمرکز روی توان عملیاتی در برنامه‌های موازی است.

## ۳ محاسبات ناهمگون یا HETEROGENEOUS

در روزهای آغازین دنیای کامپیوتر، کامپیوترها تنها شامل واحد پردازنده مرکزی بودند یعنی **CPU** ها که به منظور اجرای کارهای برنامه‌نویسی کلی طراحی شده بودند. ولی در دهه‌ی گذشته، کامپیوترهای پیشرو در محاسبات با کارآیی بالا، برخی المان‌های پردازشی دیگری را نیز استفاده کردند. شاخص‌ترین آن **GPU** هست، که ذاتا برای انجام محاسبات گرافیکی به صورت موازی طراحی شده است. در طی زمان، **GPU** ها قوی‌تر و جامع‌تر شده، و باعث شد که بتوان از آن‌ها برای مقاصد کلی در کارهای محاسباتی استفاده کرد یعنی کارهای **general-purpose** که این کار با یک کارآیی فوق العاده و توان مصرفی بهینه انجام می‌شود.

معمولا، **CPU** ها و **GPU** ها به عنوان واحدهای پردازشی مجزایی هستند که با باس **PCI-Express** و تنها یک نود محاسباتی به هم متصل می‌شوند. در این معماری، **GPU** ها را به عنوان یک دیوایس مجزا می‌بینند.

تغییراتی که منجر به استفاده از سیستم‌های ناهمگون به جای سیستم‌های همگون شد راهی بود که برای رسیدن به محاسبات با کارآیی بالا در تاریخ طی شد. سیستم‌های همگون از یک یا چندین پردازنده از یک معماری یکسان برای اجرای یک برنامه استفاده میکردند. سیستم‌های ناهمگون از معماری**های** مناسب برای اجرای برنامه استفاده می‌کنند، و هر کار متناسب با معماری به پردازنده تخصیص داده می‌شود، که منجر به افزایش کارآیی در این گونه سیستم‌ها می‌شود.

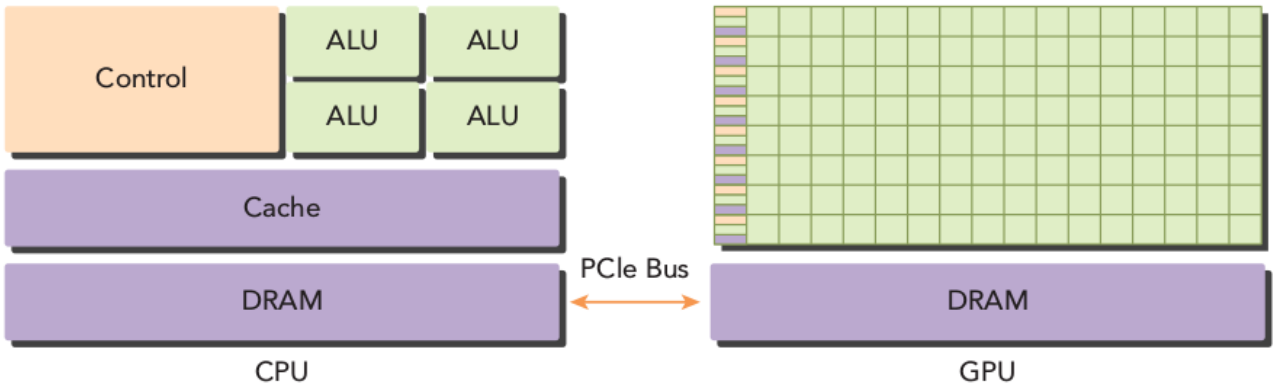
اگر چه سیستم‌های ناهمگون مزیت‌های خیلی مهمی نسبت به سیستم‌های قدیمی در محاسبات دارند، ولی امروزه استفاده موثر از این سیستم‌ها با افزایش پیچیدگی طراحی برنامه‌ها محدود شده است. در حالی که برنامه‌نویسی موازی توجه‌ها را به خود جلب کرده است، همچنان پیچیدگی‌هایی نیز در خود گنجانده است.

اگر شما در زمینه‌ی برنامه‌نویسی موازی تازه‌کار هستید، در این صورت می‌توانید از ابزارهای پیشرفته و توسعه‌هایی که در زمینه‌ی سیستم های ناهمگون رخ داده است استفاده لازم را ببرید. اگر که یک برنامه‌نویس سطح بالا در زمینه‌ی پردازش موازی هستید، تطابق یافتن با معماری ناهمگون برای شما آسان‌تر خواهد بود.

## ۴ معماری heterogeneous یا نامتجانس (ناهمگون)

امروزه، نودهای محاسباتی در یک معماری عادی ناهمگون متشکل از دو سوکت **CPU** و دو یا بیشتر **GPU** است. یک **GPU** دیگر یک پلتفرم مستقل نیست بلکه یک پردازنده کمکی برای **CPU** می‌باشد. بنابراین، **GPU** ها باید به عنوان یک دیوایس برای **CPU** با استفاده از یک باس **PCI-Express** باشند، که

در شکل زیر نمایش داده شده است. دقیقا به همین دلیل است که، در اصطلاح برنامه‌نویسی **GPU** مـــا از **CPU** به عنوان **host** و از **GPU** به عنوان **device** استفاده می‌کنیم.




در **heterogeneous** ها برنامه از دو بخش تشکیل میگردد.

- کد میزبان **(Host code)**
- کد دیوایس **(Device code)**

**Host code** بر روی **CPU** ها اجرا شده و **device code** بر روی **GPU** ها اجرا می‌شود. یک برنامـه کـه بـر روی یک پلتفرم **heterogeous** اجرا می‌شود، معمـولا بـا **CPU** شـروع می‌شـود. کـد **CPU** مسـوولیت‌های روبه‌رو را دارد: مدیریت محیط، کد، و داده‌ای است که دیوایس میگیرد.

با برنامه‌های محاسباتی سنگین، معمولا **program section** مقدار قابل توجهی از داده‌هـای مـوازی را در خود خواهد داشت. **GPU** ها برای سرعت بخشیدن به اجرای این قسمت از داده‌ی موازی هسـتند. وقـتی که یک قطعه‌ی سخت‌افزاری که به صورت سخت‌افزاری نـیز از **CPU** جـدا می‌باشـد بـه منظـور سـرعت بخشیدن به این قسمت‌ها استفاده شود، معمولا از آن به عنوان یک شتاب دهنده یاد می‌شود. **GPU** ها به عنوان بارزترین شتابدهنده‌ی سخت‌افزاری می‌باشد.

پلتفرم محاسباتی **NVIDA's GPU** در سری‌های زیر فعال هستند:

- Tegra
- GeForce
- Quadro
- Tesla

خانواده‌ی **Tegra** به منظور استفاده در موبایل‌ها و ابزارهای نهفته مانند تبلت‌ها و گوشی‌ها طراحی شده، **Gefore** برای اسـتفاده کـاربران، **Quadro** بـرای مصورسـازی‌های حـرفه‌ای و **Tesla** بـرای دیتاسـنترهای محاسبات موازی هستند. **Fermi**، شتاب‌دهنده‌ی **GPU** از خانواده‌ی **Tesla**، اخـیرا توجهـات زیـادی را بـه عنوان شتاب‌دهنده‌ی محاسباتی در برنامه‌های با کارآیی بالا جلب کـرده اسـت. **Fermi**، توسـط **NVIDIA** در سال ۲۰۱۰ به عنوان اولین معماری کامل محاسباتی **GPU** رونمایی شد. **GPU** شـتاب‌دهنده‌ی **Fermi**

تـا بـه امـروز در بسـیاری از زمینه‌هـای محاسـباتی اسـتفاده شـده اسـت مثـل مطالعـات زمین‌لـرزه، شبیه‌سازی‌های بیوشیمی، مدل‌سازی آب و هوا، و پردازش سیگنال، محاسبات مالی، آنـالیز داده. **Kepler**، نسل فعلی معماری **GPU** که بعد از **Fermi** بود، در پاییز ۲۰۱۲ رونمایی شد، که قـدرت پردازشـی خیلی بیشتری را نسبت به نسل قبلی پیشنهاد می‌کرد و روش‌های جدیدی برای بهبود و بهینه کردن اجراهـای موازی در اختیار قرار میداد. **Tegra K1** از **Kepler** به منظور افزایش قدرت محصولات نهفته استفاده کرده است.

ویژگی‌های مهمی وجود دارد که مشخص کننده‌ی توانایی **GPU** می‌باشد:

- تعداد هسته‌های **CUDA**
- میزان حافظه

همچنین، دو متریک برای اندازه‌گیری کارآیی یک **GPU** وجود دارد:

- **peak computational performance**
- **memory bandwidth**

**peak computational performance** راهی برای اندازه‌گیری قابلیت محاسباتی است، که به این صـورت تعریف می‌شود که چقدر محاسبات ممیز شناور **single** یا **double** را می‌توان در یک ثانیه انجام داد. **Peak performance** معمولا به صورت **gflops** و یا **tflops** بیان می‌شود. پهنای باند حافظه با **gigabytes/sec** بیان می‌شود. جدول زیر یک خلاصه‌ای از معماری **Fermi** و **Kepler** و ویژگی‌های کارآیی را آورده است.

| | FERMI (TESLA C2050) | KEPLER (TESLA K10) |
|---|---|---|
| CUDA Cores | 448 | 2 x 1536 |
| Memory | 6 GB | 8 GB |
| Peak Performance* | 1.03 Tflops | 4.58 Tflops |
| Memory Bandwidth | 144 GB/s | 320 GB/s |

* Peak single-precision floating point performance

بیشتر مثال‌هایی که در این کتاب زده می‌شود هم بر روی **Fermi** قابـل اجـرا خواهـد بـود و هم بـر روی **Kepler**. برخی از مثال‌ها نیازمند یک معماری خاص می‌باشد کـه تنهـا در **GPU** هـای **Kepler** گنجانـده شده است.

## ۵ قابلیت محاسباتی یا compute capabilitiy

**NVIDIA** از یک اصطلاح خاص یعنی **compute capability** برای ورژن سخت‌افزار **GPU** برای همه‌ی خانواده‌ی **Tesla** استفاده می‌کند. دیوایس‌ها با ورژن کلی یکسان معماری هسته یکسانی خواهند داشت.

- معماری کلاس **Kepler** با عدد ۳ نمایش داده می‌شود.
- معماری کلاس **Fermi** با عدد ۲ نمایش داده می‌شود.
- معماری کلاس **Tesla** با عدد ۱ نمایش داده می‌شود.

اولین کلاس **GPU** هایی که توسط **NVIDIA** رونمایی شد، در خود نام **Tesla** را برای همه‌ی **GPU** ها نیز داشت (یعنی هم نام **Tesla** بود و هم معماری سخت‌افزار هسته **Tesla** بود). همه‌ی مثال‌های این کتاب نیاز به **cp** بالای ۲ خواهند داشت.

TABLE 1-2: Compute Capabilities of Tesla GPU Computing Products

| GPU | COMPUTE CAPABILITY |
|---|---|
| Tesla K40 | 3.5 |
| Tesla K20 | 3.5 |
| Tesla K10 | 3.0 |
| Tesla C2070 | 2.0 |
| Tesla C1060 | 1.3 |

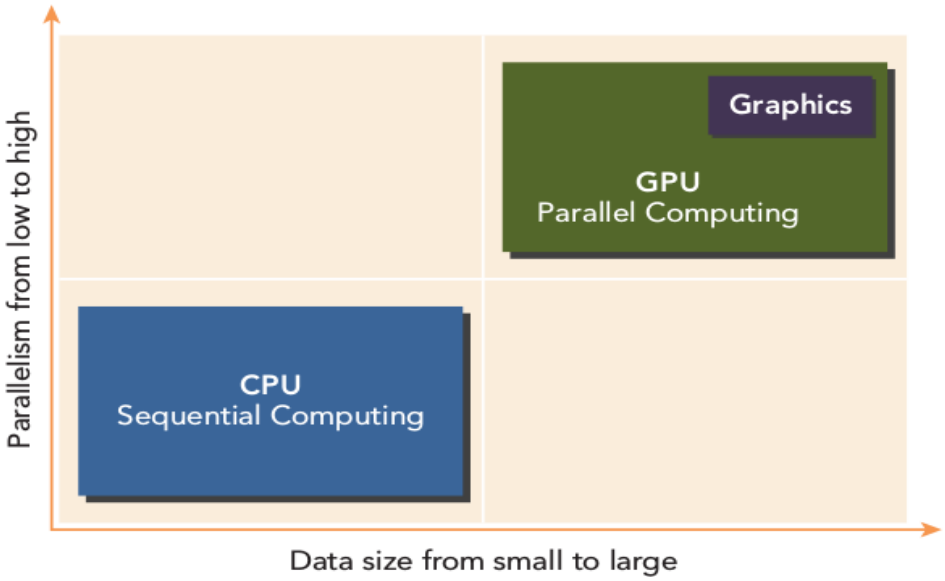
## ۶ ۶ الگوی محاسباتی ناهمگون

محاسبات **GPU** قصد جایگزینی با محاسبات **CPU** را ندارند. هر کدام مزیت‌های خاصی برای نوع خاصی از برنامه‌ها دارند. محاسبات **CPU** برای عملیات‌هایی که کنترل سختی دارند مناسب می‌باشد، و **GPU** برای محاسباتی مفید هستند که در آن **Task** ها محاسبات موازی زیادی دارند. وقتی **CPU** همراه با **GPU** قرار میگیرد، یک ترکیب خیلی قوی را ایجاد خواهند کرد. **CPU** برای محاسبات کوتاه که کنترل غیر قابل پیش‌بینی دارند مناسب می‌باشد، و **GPU** برای محاسبات سنگین ولی کنترل سبک مناسب می‌باشد. در واقع دو پارامتر برای انتخاب بهتر بین **CPU** و **GPU** وجود دارد:

- سطح موازی سازی
- اندازه داده

اگر یک مساله حجم داده کمی داشته باشد، و منطق برنامه پیچیده باشد، و یا سطح موازی‌سازی پایینی داشته باشد، در این صورت **CPU** انتخاب مناسب‌تری می‌باشد چرا که توانایی مناسبی در منطق پیچیده‌تر دارد. اگر یک برنامه دارای مقدار زیادی داده بود، و سطح موازی‌سازی بالایی داشته باشد، در

این صورت **GPU** انتخاب مناسب تری می‌باشد چرا که **GPU** تعداد هسته‌های بسیار بیشتری دارد، که می‌تواند از **multi-threading** به صورت گسترده استفاده کند، و نسب به **CPU** دارای عرض پهنای باند بیشتری می‌باشد.




معماری محاسبات موازی ناهمگون **CPU+GPU** به این دلیل به وجود آمد که **CPU** و **GPU** هر کدام ویژگی‌های مکملی دارند که باعث می‌شود برنامه‌ها برای هر چیزی مهیا باشند. بنابراین، به منظور کارآیی بهینه ممکن است که نیاز داشته باشید برای برنامه خودتان همه از **CPU** و هم از **GPU** استفاده کنید، اجرای قسمت ترتیبی و یا قسمت موازی بر روی **CPU** و قسمتی که دارای داده‌ی موازی سنگین هست بر روی **GPU** که در شکل زیر نمایش داده شده است.

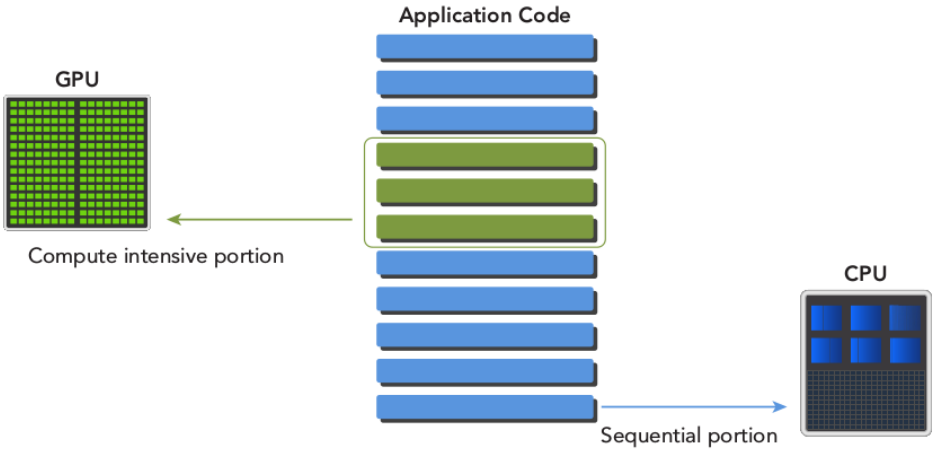



FIGURE 1-11

نوشتن کد به این صورت به شما این اطمینان را میدهد که ویژگی‌های **GPU** و **CPU** مکمل همدیگر باشند، و منجر به استفاده‌ی حداکثری از قدرت **CPU+GPU** خواهد شد. برای این که بتونیم یک برنامه

طراحی کنیم که از ترکیب **CPU+GPU** پشتیبانی داشته باشد، **NVIDIA** یک مدل برنامه‌نویسی به نام **CUDA** ایجاد کرده است. این مدل برنامه‌نویسی جدید تمرکز کتاب در ادامه خواهد بود.

تفاوت ترد **CPU** با ترد **GPU**

تردها روی **CPU** به عنوان موجودیت‌های سنگینی شناخته می‌شوند. سیستم عامل باید یک ترد را خاموش و روشن کند تا توانایی **multi-threading** را فعال کند. **Context switches** کند و پر هزینه هستند.

تردها بر روی **GPU** خیلی سبک هستند. در یک سیستم عادی، ده ها هزار ترد در صف هستند تا کاری را انجام دهند. اگر یک **GPU** باید منتظر گروهی از تردها باشد، خیلی راحت می‌تواند یک کار دیگر را اجرا کند.

هسته‌های **CPU** طراحی شده‌اند تا میزان **latency** را کمینه کنند، در حالی که هسته‌های **GPU** برای تعداد زیادی ترد موازی طراحی شده‌اند تا قابلیت محاسباتی را بیشینه کنند.

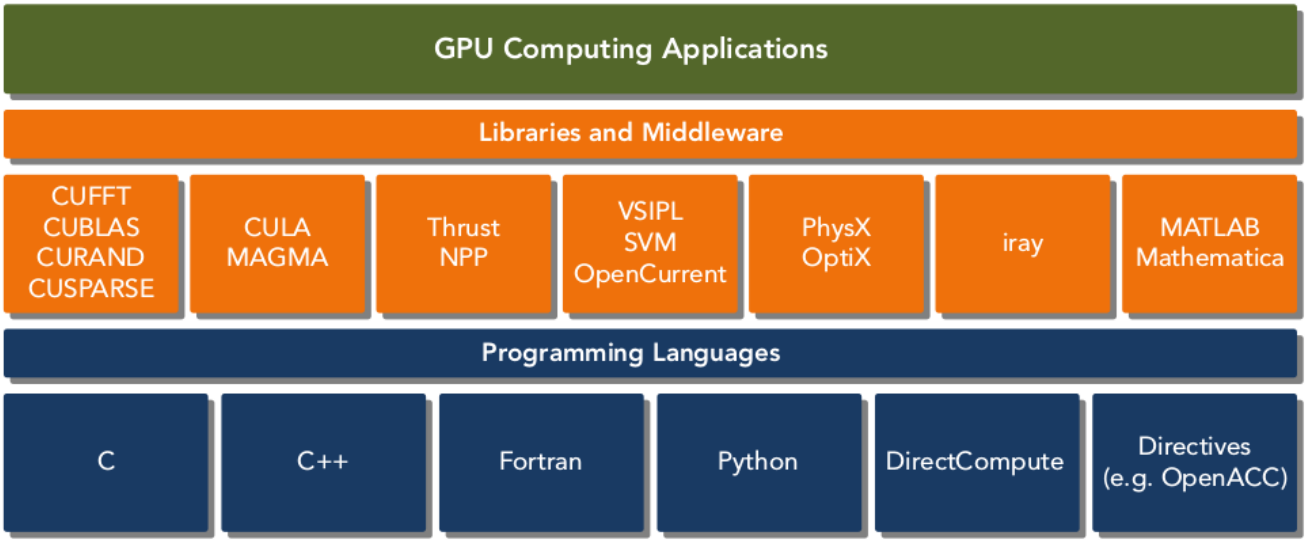
امروزه، یک **CPU** با چهار هسته **quad** می‌تواند تنها ۱۶ تا ترد را به صورت همزمان اجرا کند، البته اگر **CPU** بتواند از **hyper-threading** پشتیبانی کند می‌توان آن را تا ۳۲ تا ترد افزایش داد.

**GPU** های مدرن **NVIDIA** می‌توانند تا ۱۵۳۶ تا ترد فعال به ازای هر **multiprocessor** داشته باشند. روی یک **GPU** با ۱۶ تا **multiprocessor**، این عدد تا ۲۴۰۰۰ ترد فعال افزایش می‌یابد.

## ۷ CUDA : یک پلتفرم برای محاسبات ناهمگون

**CUDA** یک پلتفرم محاسبات موازی **general-purpose** می‌باشد، یک مدل برنامه‌نویسی که موتورهای برنامه‌نویسی موازی در **GPU** های **NVIDIA** را به منظور حل مسایل پیچیده به صورت موثری به کار میگیرد. با استفاده از **CUDA** ، شما می‌توانید به **GPU** به منظور محاسبات دسترسی داشته باشید، که قبلا بر روی **CPU** اجرا میشد.

پلتفرم **CUDA** از طریق کتابخانه‌های شتابدهنده‌ی **CUDA** ، رابط برنامه‌نویسی و افزونه‌هایی که برای زبان‌های استاندارد فراهم شده شامل **C,C++,Fortran** و **python** قابل دسترسی خواهد بود. این کتاب بر روی **CUDA C programming** تمرکز خواهد داشت.

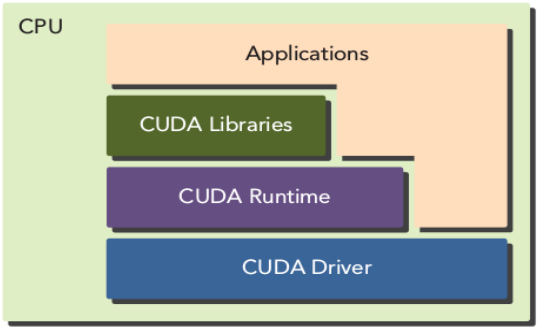



**CUDA C** یک افزونه از **ANSI C** استاندارد می‌باشد که افزونه‌های زبان به درد بخوری برای فعال کردن برنامه‌نویسی ناهمگون در اختیار ما قرار میدهد، همچنین یک **API** سر راست برای مدیریت **device** ، حافظه، و **task** های دیگر نیز دارد. **CUDA** همچنین یک مدل برنامه‌نویسی **scalable** دارد که می‌شود میزان **scalable** بودن موازی سازی را با تغییر تعداد هسته‌ها تغییر داد، در عین حال نیازی به معلومات پیچیده در مورد **GPU** نخواهد بود و برای کسی که زبان **C** را یاد گرفته این کار آسان خواهد بود.

**CUDA** دو سطح **API** برای مدیریت **GPU device** و مدیریت تردها فراهم کرده است که بعدا در شکل نیز خواهیم دید:

- CUDA Driver API
- CUDA Runtime API

**API** مربوط به **driver** یک **API** سطح پایین بوده و تقریبا برای برنامه‌نویسی دشوار می‌باشد، اما کنترل بیشتری بر روی **GPU** در اختیار ما قرار میدهد. **API** مربوط به **runtime** یک **API** سطح بالا بوده که بر روی **API** درایور ساخته شده است. هر تابع از **API** مربوط به **runtime** به عملیات‌های ابتدایی تر که مربوط به **API** درایور است شکسته می‌شود.

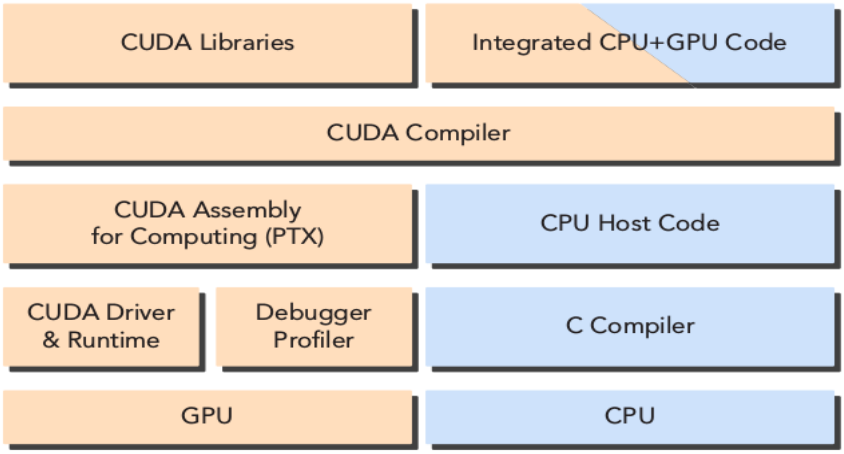


**در حقیقت در مورد کارآیی این دو API تفاوت زیادی وجود ندارد. این که چطور کرنل شما از حافظه استفاده می‌کند و چطور تردها را مدیریت می‌کنید چیزی است که تفاوت کارآیی را مشخص می‌کند.**
**این دو API دو به دو با همدیگر ناسازگار هستند. شما باید یکی از این دو را انتخاب کنید، و این امکان وجود ندارد که توابع این دو را با هم ترکیب کنید. همه‌ی مثال‌ها در این کتاب از runtime استفاده می‌کنند.**

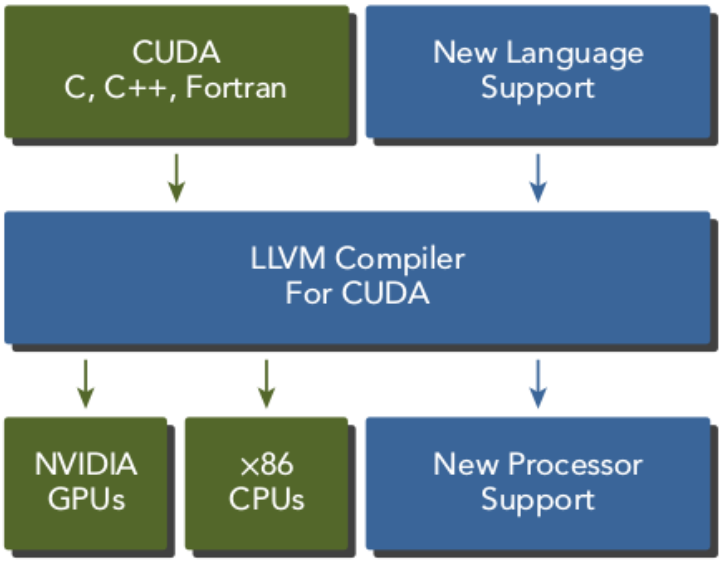
یک برنامه **CUDA** ترکیبی از دو بخش زیر می‌باشد.

- یک کد میزبان که بر روی **CPU** اجرا می‌شود.
- یک کد **device** که بر روی **GPU** اجرا می‌شود.

کامپایلر **nvcc** مربوط به کودای **NVIDIA** در طی مرحله کامپایل کد **device** را از کد میزان جدا می‌کند. **Host code** در واقع زبان **C** استاندارد بوده و در ادامه توسط کامپایلر زبان **C** کامپایل خواهد شد. کد **device** با استفاده از کلمات کلیدی برای مشخص کردن توابع **data-parallel** جدا می‌شود که **kernel** نامیده می‌شود. در ادامه کد **device** بر روی **nvcc** کامپایل می‌شود. در مرحله **link**، کتابخانه‌ی **CUDA runtime** به فراخوان‌های **kernel** اضافه شده و به طور صریح بر روی **GPU** می‌تواند اجرا شود.




کامپایلر **nvcc** عمدتا بر اساس ;کامپایلر **LLVM** نوشته شده است. شما می‌توانید با استفاده از **CUDA Compiler SDK** یک زبان برنامه‌نویسی بسازید و یا توسعه دهید که از شتابدهنده‌ی **GPU** پشتیبانی کند.




The CUDA platform is also a foundation that supports a diverse parallel computing ecosystem, as shown in Figure 1-16. Today, the CUDA ecosystem is growing rapidly as more and more companies provide world-class tools, services, and solutions. If you want to build your applications on GPUs,the easiest way to harness the performance of GPUs is with the CUDA Toolkit ( https://developer.nvidia.com/cuda-toolkit ), which provides a comprehensive development environment for C and C++ developers. The CUDA Toolkit includes a compiler, math libraries, and tools for debugging and optimizing the performance of your applications. You will also fi nd code samples,programming guides, user manuals, API references, and other documentation to help you get started.

اولین کدمان را به صورت زیر می‌نویسیم.

```
#include <stdio.h>
#include <iostream>
using namespace std;
__global__ void hellofromGPU(void)
{
    printf("Hello world from GPU!! \n");
}
int main()
{
    cout<<"Hello world from CPU!!"<<endl;
    hellofromGPU<<<1,1>>>();
    cudaDeviceReset();
    return 0;
}
```

در این کد ما تابع **hellofromGPU** را به صورت زیر تعریف کرده‌ایم.

```
__global__ void hellofromGPU(void)
{
   printf("Hello world from GPU!! \n");
}
```

این تابع یک تابع کرنل است که در آن **__global__** یک **qualifier** به حساب می‌آید. این **qualifier** به کامپایلر می‌گوید که این تابع از روی **CPU** خوانده می‌شود و روی **GPU** اجرا می‌شود. برای اجرا کردن تابع کرنل باید به صورت زیر عمل کنیم.

```
hellofromGPU<<<1,10>>>();
```

در این کد **triple angle bracket** یک فراخوانی از ترد میزبان به کدی که بر روی **device** قرار دارد مارک می‌کند. یک کرنل توسط آرایه‌ای از تردها اجرا شده و همه‌ی تردها یک کد یکسان را اجرا خواهند کرد. پارامترهایی که درون براکت سه‌گانه آورده شده است، پارامترهای تنظیمی برای اجرا می‌باشد، که مشخص کننده‌ی تعداد تردهایی است که تابع کرنل را اجرا می‌کنند. در این مثال، شما از ۱۰ ترد برای اجرای کرنل استفاده می‌کنید. و خروجی به صورت زیر خواهد بود.

```
Hello world from CPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
```

تابع **cudaDeviceReset()** صریحا همه‌ی منابعی که این پردازش از **device** گرفته را تخریب و پاک می‌کند. برای کامپایل بر روی معماری **sm_20** به صورت زیر نیز می‌توانیم عمل کنیم.

```
nvcc -arch sm_20 hello.cu -o hello
```

یک برنامه‌ی عادی کودا شامل پنج قدم خواهد بود.

۱. اختصاص دادن حافظه‌ی **GPU**

۲. کپی کردن داده از روی حافظه‌ی **CPU** روی حافظه‌ی **GPU**.

۳. فراخوانی کردن یک تابع کرنل کودا برای انجام یک محاسبه‌ی خاص.

۴. کپی کردن داده از حافظه‌ی **GPU** روی حافظه‌ی **CPU**.

۵. نابود کردن حافظه‌ای که به **GPU** اختصاص داده بودیم.

## ۸ آیا برنامه‌نویسی با CUDA مشکل است؟

تفاوت عمده بین برنامه‌نویسی **CPU** و برنامه‌نویسی **GPU** در میزان آشنایی است که با ویژگی‌های معماری **GPU** باید کسب کرد. فکر کردن به صورت موازی و داشتن یک درک اولیه از معماری **GPU** به شما این امکان را میدهد که برنامه‌های موازی خودتان را بنویسید .

اگر می‌خواهید که به عنوان یک مهندس برنامه‌های موازی یک کد موثر و با سرعت بالا بنویسید، نیاز به یک آشنایی اولیه با معماری **CPU** نیز خواهید داشت. به طور مثال، **locality** یک مفهوم خیلی مهم در برنامه‌نویسی موازی است. در واقع **Locality** اشاره به استفاده مجدد از داده‌ها برای کاهش تاخیر دسترسی به حافظه می‌باشد. **Temporal Locality** یعنی این که داده و یا منابع به مدت زمان کوتاهی قابل استفاده مجدد باشد و **Spatial Locality** یعنی این که داده‌های یک المان در حافظه نزدیک به همدیگر باشند. **CPU** های مدرن از کش‌های بزرگ استفاده می‌کنند، تا برنامه‌هایی که **Spatial** و **Temporal locality** مناسبی دارند در بهترین حالت خود اجرا شوند. در واقع این مسوولیت برنامه‌نویس است که برنامه را طوری بنویسید که به بهترین نحو از **CPU cache** استفاده بکند. برنامه‌نویسان باید بهینه‌سازی‌های سطح پایین **cache** را در دست بگیرند، ولی نیازی به این نیست که در مورد تردهای **CPU** و چگونگی اجرای این تردها فکر کنند چون که **CPU** همچین اطلاعاتی را به ما نمیدهد.

**CUDA** در این مورد هم سلسله مراتب حافظه و هم سلسه مراتب تردها را در اختیار ما قرار میدهد، و توانایی کنترل اجرای تردها و برنامه‌ریزی تردها را به درجه‌ی بالاتری می‌رساند.

به طور مثال، ما یک حافظه‌ی خاص داریم که **shared memory** نامیده می‌شود، که توسط برنامه‌نویسی **CUDA** قابل کنترل است. **Shared memory** را می‌توان به عنوان کش قابل کنترل توسط برنامه دید، که افزایش سرعت قابل ملاحظه‌ای را با حفظ کردن پهنای باند **main memory** به ما میدهد. با استفاده از حافظه **share** شده، می‌توانید **locality** کدتان را مستقیما کنترل کنید.

وقتی که یک برنامه توسط **ANSI C** نوشته می‌شود، شما باید صریحا تردهای خودتان را با **pthreads** و یا **OpenMP** کنترل کنید، این دو تکنیک‌های شناخته شده‌ای هستند که برنامه‌نویسی موازی را در اکثر معماری‌ها و سیستم عامل‌ها پشتیبانی می‌کند. وقتی که یک برنامه در **CUDA C** نوشته می‌شود، در واقع یک تکه کد سریال نوشته‌اید که توسط یک ترد اجرا می‌شود. **GPU** این کرنل را گرفته و آن را با اجرا کردن بر روی هزاران ترد تبدیل به یک کد موازی می‌کند، در واقع همه‌ی تردها یک کد را اجرا می‌کنند.

مدل برنامه‌نویسی موازی **CUDA** راهی برای کنترل تردهـای معمـاری میدهـد. دسـتکاری مسـتقیم این سازماندهی روی ترتیب تردهایی که در **GPU** اجرا می‌شود تاثیر میگذارد. از آنجایی کـه **C CUDA** یـک افزونه از زبان **C** است، معمولا تبدیل کد زبان **C** به **CUDA C** کار سختی نخواهد بود.

**CUDA** جزییات سخت‌افزاری را خود حل می‌کند و نیازی نیست که برنامه به **API** سابق نگاشت شود. در هسته‌ی **CUDA** ما سه نوع مفهوم ابسترکت شده داریم: سلسله مراتب گروه‌های ترد، یک سلسله مـراتب از گروه‌های حافظه، و یک حایل سنکرون سازی، که به صورت حداقل چیزی که از افزونه‌هـا نیـاز داریـد ارایه شده است. با هر نسخه‌ی **CUDA**، برنامه‌نویسی موازی توسط **NVIDIA** سـاده‌تر شـده. البتـه بـرخی هنوز مفاهیم **CUDA** را سطح پایین می‌دانند، و بالا بردن این ابسترکت ممکن است که توانـایی شـما در کنترل برنامه تان را کاهش دهد. بدون این کنترل، کارآیی برنامه شما دیگر در کنترل شما نخواهـد بـود. بنابراین، چالشی که باید با آن روبه‌رو شوید آشنایی اولیه بـا معمـاری **GPU** و خـبره شـدن در ابزارهـای توسعه **CUDA** و محیط توسعه می‌باشد.

## ۹ محیط توسعه‌ی CUDA

**NVIDIA** محیط توسعه‌ی بزرگی را برای توسعه برنامه‌هایی که نیاز به **GPU** دارند، توسط **C** و **C++** آماده کرده است که شامل:

- NVIDIA NsightTM integrated development environment
- CUDA-GDB command line debugger
- Visual and command line profiler for performance analysis
- CUDA-MEMCHECK memory analyzer
- GPU device management tools

پس از آشنایی با این ابزارها، برنامه‌نویسی با زبان **C** یک کار آسان خواهد بود.

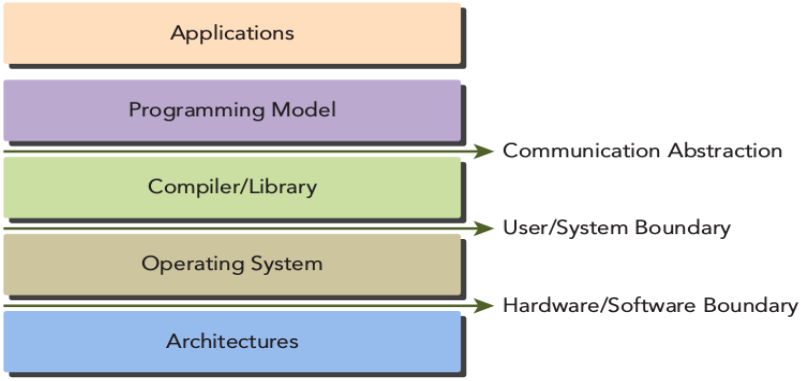
***Locality* in this sense is speaking mostly to *cache locality*. Writing data structures and algorithms to operate mostly out of cache makes the algorithm run as fast as it possibly can. Cache locality is one of the reasons *quick sort* is quick.**
**For a data structure, you want to keep the parts of your data structure that refer to each other relatively close to each other, to avoid flushing out useful cache lines.**
**Also, you can rearrange your data structure so that the compiler will use the minimum amount of memory required to hold all the members and still efficiently access them. This helps make sure your data structure consumes the minimum number of cache lines.**
**A single cache line on a current x86-64 architecture (core i7) is 64 bytes.**

# فصل دوم : مدل برنامه‌نویسی CUDA

**CUDA** یک پلتفرم محاسبات موازی و یک مدل برنامه‌نویسی است که همراه با مجموعه‌ی کوچکی از افزونه‌ها است که به زبان **C** افزوده شده. با استفاده از **CUDA**، می‌توانید به سادگیه نوشتن یک برنامه‌ی زبان **C**، کد نویسی کنید. با استفاده از **CUDA** می‌توانید برای هزاران سیستم که از کارت گرافیک **NVIDIA** استفاده می‌کنند برنامه بنویسید، که محدوده‌ای از دیوایس‌های نهفته، تبلت‌ها، لپ‌تاپ‌ها، سیستم‌های دسکتاپ تا **workstation** برای سیستم‌های کلاستر **HPC** را شامل می‌شود. ابزارهای برنامه‌نویسی مشابه با زبان **C** توسعه داده شده اند تا در امر تصحیح، دیباگ و آنالیز برنامه به شما کمک کنند. در این فصل، یاد خواهید گرفت تا چطور یک برنامه **CUDA** بنویسید، این کار را با دو مثال ساده توضیح خواهیم داد: جمع برداری و جمع ماتریسی.

## ۱ معرفی مدل برنامه‌نویسی CUDA

مدل برنامه‌نویسی یک **abstraction** از معماری کامپیوتر را بیان می‌کند که به عنوان یک پل فی ما بین برنامه و پیاده‌سازی روی سخت افزار عمل می‌کند. شکل زیر لایه‌های مهم این **abstraction** را که بین برنامه و پیاده‌سازی مدل برنامه‌نویسی خوابیده است را نشان می‌دهد.




**Abstraction** ارتباط، یک مرز بین برنامه و پیاده‌سازی مدل برنامه‌نویسی است، که از طریق کامپایلر یا کتابخانه توسط سخت افزار و سیستم عامل فهمیده می‌شود. برنامه ای که توسط مدل برنامه‌نویسی نوشته می‌شود، نشان‌دهنده‌ی کامپوننت‌هایی است که برنامه به اشتراک می‌گذارد. مدل برنامه‌نویسی یک دید منطقی از معماری محاسباتی مشخصی را ارایه میدهد. معمولا، آن را در زبان برنامه‌نویسی و یا محیط برنامه‌نویسی قرار داده است.

علاوه بر این **abstraction** هایی که با سایر مدل‌های برنامه‌نویسی موازی بـه اشـتراک گذاشـته می‌شـود، مدل برنامه‌نویسی **CUDA** ویژگی‌های خاص زیر را برای دسترسی به قدرت بیشتر محاسـباتی در اختیـار قرار میدهد.

- راهی برای مدیریت تردها بر روی **GPU** از طریق یک ساختار سلسله مراتبی.
- راهی برای دسترسی به حافظه بر روی **GPU** از طریق یک ساختار سلسله مراتبی.

ما در اولین بحثی که در این فصل و فصل بعد خواهیم داشت روی مورد اول تمرکز خواهیم کرد، و مـورد دوم را در فصل چهار و پنج مورد بررسی قرار خواهیم داد.

از دید یک برنامه‌نویس، می‌توانید برنامه‌نویسی موازی را در سطوح مختلفی ببینید، مثل:

- سطح **Domain**
- سطح منطقی
- سطح سخت افزاری

وقتی که دارید برنامه‌نویسی انجام میدهید و الگوریتم را طراحی میکنیـد، توجه‌تـان معطـوف بـه **سطح domain** می‌باشد: چطور داده‌ها را تقسیم کنیم و چطور توابع رو برای حل مساله مان در حالت موازی بـه صورت کارآمدی به کار بگیریم، این **سطح domain** است. وقتی که وارد فاز برنامه نویسی شدید، باید بـه این فکر کنید که چطور تردهای موازی را مدیریت کنیـد، در طی این فـاز شـما در **سطح منطقی** فکـر می‌کنید تا اطمینان حاصل کنیـد کـه تردهـا و محاسـباتمان مسـاله را درسـت حـل می‌کننـد. در زبـان برنامه‌نویسی موازی که توسط **C** عادی انجام می‌شود، شما باید به طور صریح از **pthreads** و یا **OpenMP** استفاده کنید. **CUDA** این **abstraction** ای که در مورد کنترل رفتار تردها وجود دارد را در اختیار ما قرار داده. وقتی که مثال‌های این کتاب رو می‌بینیم، خواهید دید که چنین **abstraction** ای چه مقیاس‌پذیری فوق‌العاده‌ای را به ما می‌دهد. در **سطح سخت افزار**، قادر به درک چگونگی نگاشت شدن تردها بـر روی هسته‌ها خواهیم بود، شاید این موضوع به بهبود دادن کارآمدی سیستم کمک کند.

مدل ترد **CUDA** اطلاعات کافی را در اختیار شما قرار میدهد بدون این کـه شـما را مجبـور بـه دانسـتن جزییات سطح پایین کند. در این مورد در فصل ۳ بیشتر خواهیم دانست.

## ۲ ساختار برنامه‌نویسی CUDA

مدل برنامه‌نویسی **CUDA** به شما این امکان را میدهد که برنامه‌تان را بر روی سیستم‌های ناهمگون اجرا کنید، این کار با نشان‌دار کردن کد با یک مجموعه‌ی کوچک از افزونه‌هایی که بـه زبـان **C** افـزوده شـده انجام میپذیرد. یک محیط ناهمگون متشکل شده از **CPU** ها به همراه **GPU** می‌باشد، که هر کـدام دارای

حافظه‌ی اختصاصی است که با گذرگاه **PCI-Express** از همدیگر جدا شده‌اند. بنابراین، باید متوجه تفاوتی که این دو حافظه دارند باشید:

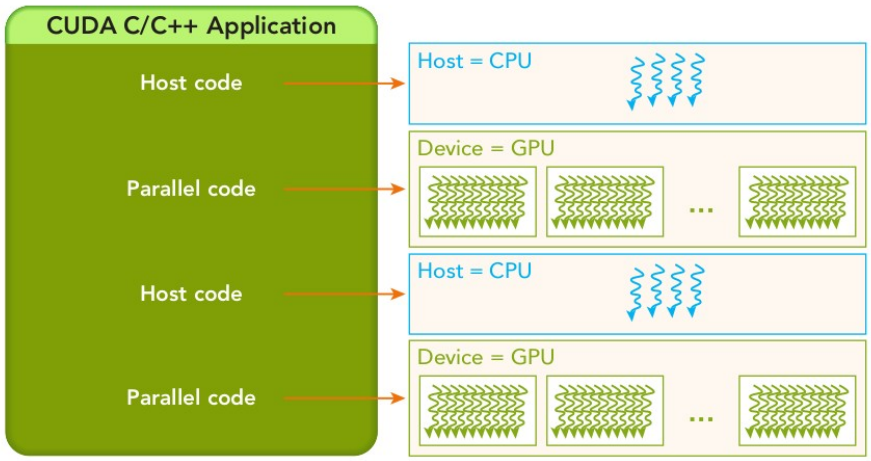
- **Host : CPU** و حافظه اختصاصی خودش( حافظه‌ی **host**)
- **Device : GPU** و حافظه‌ی اختصاصی( حافظه‌ی **device**)

برای مشخص شدن تفاوت این دو حافظه، متغیرهایی که با **_h** شروع می‌شوند برای حافظه‌ی **host** و متغیرهایی که با **_d** شروع می‌شوند برای حافظه‌ی **device** به کار می‌رود.

با شروع **CUDA 6**، **NVIDIA** یک بهبود در مدل برنامه‌نویسی داد که **Unified Memory** ( حافظه‌ی یکپارچه) نامیده می‌شود، که به عنوان یک پل بین حافظه‌ی **host** و **device** کار می‌کند. این توسعه به شما این اجازه را میدهد که هم به حافظه‌ی **CPU** و هم **GPU** با یک اشاره‌گر دسترسی داشته باشید، در حالی که سیستم به صورت اوتوماتیک داده‌ها را بین **host** و **device** جابه‌جا می‌کند. اطلاعات بیشتر در مورد این حافظه در فصل چهارم توضیح داده خواهد شد. قبل از آن شما نیاز دارید که یاد بگیرید چطور روی **host** و **device** حافظه اختصاص دهید، و صریحا داده را بین **CPU** و **GPU** جابه‌جا کنید. این کنترلی که برنامه‌نویس بر روی حافظه و داده دارد می‌تواند باعث بالابردن حداکثری کارآمدی سیستم و استفاده بیشینه از سخت افزار شود.

یک کامپوننت کلیدی در مدل برنامه‌نویسی **CUDA** کرنل می‌باشد( کرنل کدی است که بر روی **GPU** اجرا می‌شود. به عنوان یک توسعه دهنده، شما می‌توانید یک کرنل را به صورت یک برنامه‌ی ترتیبی بنویسید. در پشت صحنه، **CUDA** این کرنل نوشته شده توسط برنامه‌نویس را برای اجرا بر روی تردهای **GPU** مدیریت می‌کند. از طریق **host** ، تعریف می‌کنید که چطور الگوریتم روی **device** بر اساس داده‌هایی که دارید نگاشت شود. در این صورت ما درگیر مدیریت هزاران ترد **GPU** نخواهیم شد.

**Host** می‌تواند برای بیشتر عملیات‌ها به طور مستقل از **device** کار کند. وقتی که یک **kernel** اجرا می‌شود، کنترل بلافاصله دست **host** میفتد، و **CPU** را آزاد نگه می‌دارد تا محاسبات موازی بر روی **device** اجرا شود. مدل برنامه‌نویسی **CUDA** یک پروسه آسنکرون می‌باشد، بنابراین محاسبات **GPU** می‌تواند بر روی **GPU** با ارتباطات **host** همپوشانی پیدا کند. یک برنامه‌ی عادی **CUDA** متشکل از یک کد سریال هست که با یک کد موازی تکمیل شده است. همانطور که در شکل نمایش داده شده است، کد ترتیبی بر روی **host** اجرا می‌شود، ولی کد موازی بر روی دیوایس **GPU** اجرا می‌شود. کد **host** با استفاده از زبان **C** نوشته شده در حالی که کد **device** با استفاده از زبان **CUDA C** نوشته شده است. می‌توانید همه‌ی کد را در داخل یک سورس فایل بنویسید، یا این که می‌توانید چندین سورس فایل داشته باشد. کامپایلر **NVIDIA(NVCC)** هم برای **host** و هم برای **device** کد را تولید می‌کند.




یک روش معمول برای پردازش برنامه **CUDA** روش زیر می‌باشد.

۱. کپی کردن داده از **CPU** بر روی حافظه‌ی **GPU**

۲. فراخوانی کردن کرنل برای پردازش داده‌هایی که در **GPU** ذخیره شده است.

۳. کپی کردن حافظه از **GPU** به **CPU** .

ما کار را با مدیریت حافظه و تبادل اطلاعات بین **host** و **device** آغاز خواهیم کرد. در ادامه فصل جزییات بیشتری در مورد کرنل‌های **GPU** خواهیم دید.

## ۳ مدیریت حافظه

مدل برنامه‌نویسی **CUDA** فرض می‌کند که سیستم از یک **host** و از یک **device** تشکیل شده است، که هر کدام دارای حافظه‌ی جداگانه خود هستند. کرنل در خارج از حافظه‌ی **device** اجرا می‌شود تا این امکان را به شما بدهد که کنترل کاملی روی برنامه داشته باشید و به بهترین کارآمدی برسید، **CUDA runtime** توابعی رو برای اختصاص حافظه‌ی **device**، رها کردن حافظه‌ی **device** ، و انتقال اطلاعات بین حافظه‌ی **host** و **device** را فراهم کرده است. جدول زیر یک لیست از توابع استاندارد **C** و تابع منطبق در **CUDA C** را فراهم کرده است.

**TABLE 2-1:** Host and Device Memory Functions

| STANDARD C FUNCTIONS | CUDA C FUNCTIONS |
|---|---|
| malloc | cudaMalloc |
| memcpy | cudaMemcpy |
| memset | cudaMemset |
| free | cudaFree |

تابعی که برای اختصاص حافظه در **GPU** استفاده می‌شود **cudaMalloc** می‌باشد، و نوع استفاده از تابع به صورت زیر می‌باشد.

```
cudaError_t cudaMalloc ( void** devPtr, size_t size );
```

این تابع یک رنج خطی از حافظه **device** به میزانی که مشخص شده میگیرد. حافظه گرفته شده به بایت می‌باشد. حافظه اختصاص داده شده نیز توسط **devPtr** قابل بازیابی خواهد بود. احتمالا متوجه تشابهی که بین **cuda-malloc** و **malloc** در زبان **C** است پی برده‌اید. این تشابه، عمدی بوده. با نزدیک نگه داشتن رابط زبان **CUDA** به زبان برنامه‌نویسی **C** ، استفاده از **CUDA** آسان تر خواهد بود.

تابعی که برای انتقال داده بین **host** و **device** مورد استفاده قرار میگیرد **cudaMemcpy** می‌باشد، و نحوه‌ی استفاده از این تابع نیز به صورت زیر می‌باشد.

```
cudaError_t cudaMemcpy(void* dst,const void* src,size_t count,cudaMemcpyKind kind);
```

این تابع به میزان بایت‌های مشخص شده از قسمت حافظه‌ی سورس، که توسط **src** به آن اشاره شده، به فضای حافظه‌ی مقصد که با **dst** به آن اشاره شده یک کپی انجام میدهد. توجه شود که نوع **cudaMemcpyKind** به چهار دسته زیر تقسیم می‌شود:

- cudaMemcpyHostToHost
- cudaMemcpyHostToDevice
- cudaMemcpyDeviceToHost
- cudaMemcpyDeviceToDevice

این تابع یک رفتار آسنکرون نخواهد داشت، چون برنامه‌ی **host** تا وقتی که تابع کارش تمام نشود بلاک می‌ماند. هر فراخوانی از **CUDA**، به جز فراخوانی کرنل، یک کد خطا به صورت **enumerated** شده در **cudaError_t** برمیگرداند. به طور مثال،اگر حافظه‌ی **GPU** به درستی تخصیص داده شده باشد، چیزی که این تابع برمیگرداند به صورت زیر خواهد بود.

**cudaSuccess**

در غیر این صورت چیزی که برمیگرداند به صورت زیر هست.

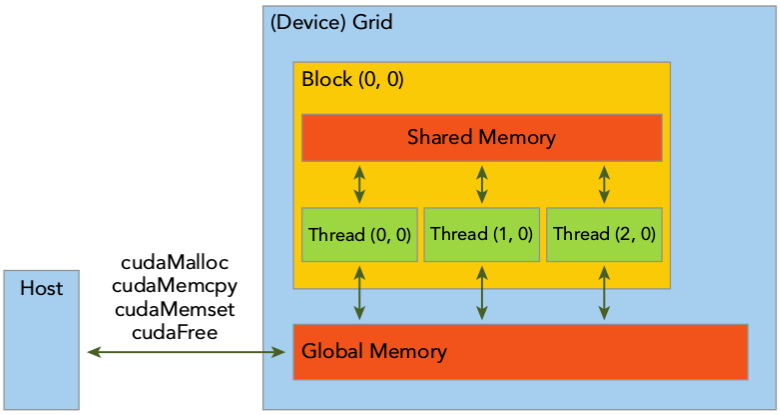
**cudaErrorMemoryAllocation**

برای تبدیل این کد خطا به چیزی که برای انسان قابل خواندن باشد می‌توانید از این تابع **run-time** **cudaGetErrorString** استفاده کنید.

```
const char* string=cudaGetErrorString(cudaError_t   Error);
```

تابع **cudaGetErrorString** مشابه تابع **strerror** در زبان **C** استاندارد می‌باشد.

مدل برنامه‌نویسی **CUDA** یک **abstraction** از سلسله مراتب حافظه از معماری **GPU** در اختیار میگذارد. شکل زیر یک ساختار ساده شده حافظه‌ی **GPU** را نشان میدهد، که شامل دو زیر شاخه‌ی مهم است: حافظه‌ی **global** و حافظه‌ی **share** شده. در مورد ساختار سلسله مراتبی **GPU** در فصل چهار و پنج بیشتر یاد خواهیم گرفت.

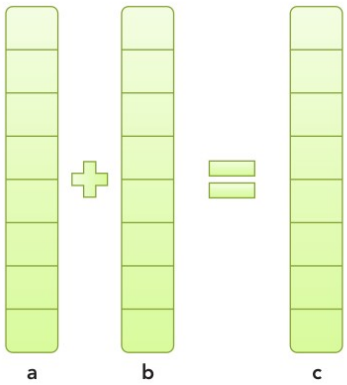



## ۳ سلسله مراتب حافظه **GPU** (خلاصه)

یکی از ویژگی‌های خیلی خاص برنامه‌نویسی **CUDA** اطلاعاتی است که در مورد سلسله مراتب حافظه در اختیار ما قرار داده شده. هر **GPU** دارای مجموعه حافظه‌ی متفاوتی می‌باشد که برای کاربردهای متفاوت انتخاب شده است. در مورد جزییات این بحث در فصل چهار و پنج بیشتر بحث خواهیم کرد.

در سلسله مراتب حافظه‌ی **GPU**، دو حافظه‌ی مهم وجود دارد که به عنوان حافظه‌ی **global** و **shared** شناخته می‌شود. حافظه‌ی **global** مشابه حافظه‌ی **CPU** بوده، در حالی که **shared memory** شبیه حافظه‌ی کش **CPU** می‌باشد. ولی با این تفاوت که، حافظه‌ی **shared** را می‌توان به طور مستقیم با کرنل کنترل کرد.

فعلا قرار هست که به مدیریت داده بین **host** و **device** بپردازیم، و یاد بگیریم که چطور از طریق **CUDA C** دو آرایه را با همدیگر جمع کنیم. همانطور که در شکل بعدی مشخص است، المان اول از آرایه‌ی **a** با المان اول از آرایه‌ی **b** جمع می‌شود، و نتیجه در المان اول در آرایه‌ی **c** گذاشته می‌شود. این محاسبات برای همه‌ی المان‌ها تکرار می‌شود.




برای جمع زدن دو آرایه ابتدا با **host code** شروع می‌کنیم(**C** استاندارد).

```
#include <stdlib.h>
#include <string.h>
#include <time.h>
void sumArrayOnHost(float *A,float *B,float *C,const int N)
{
    for (int idx = 0; idx <N; ++idx)
    {
        C[idx]=A[idx] + B[idx];
    }
}
void initialData(float *ip,int size)
{
    //generate different seed for random number
    time_t t;
    srand((unsigned int) time(&t));
    for (int i = 0; i < size; ++i)
    {
        ip[i] = (float) (rand() & 0xFF) /10.0f;
    }
}
int main()
{
    int nElem=1024;
    size_t nBytes = nElem * sizeof (float);
    float *h_A,*h_B,*h_C;
    h_A = (float *)malloc(nBytes);
    h_B = (float *)malloc(nBytes);
```

```
    h_C = (float *)malloc(nBytes);
    initialData(h_A, nElem);
    initialData(h_B, nElem);
    sumArrayOnHost(h_A, h_B, h_C,nElem);
    free(h_A);
    free(h_B);
    free(h_C);
    return(0);
}
```

از آنجایی که این یک برنامه خالص زبان **C** هست، می‌تونید با کامپایلر زبان **C** کامپایلش کنید. همچنین می‌تونید با استفاده از **nvcc** به صورت زیر کامپایلش کنید.

```
$ nvcc -Xcompiler -std=c99 sumArraysOnHost.c –o sum
$ ./sum
```

کامپایلر **CUDA** به شما این امکان را میدهد که آپشن‌های فراوانی را به طور مستقیم به **nvcc** پاس دهید. فلگ **Xcompiler**- آپشن‌ها را مستقیما برای کامپایلر **C** مشخص می‌کند. در این مثال، **std=c99**- به کامپایلر پاس داده شده است چون کد نوشته شده طبق استاندارد **C99** نوشته شده است. شما می‌توانید این آپشن‌ها را در داکیومنت کامپایلر **CUDA** در آدرس زیر پیدا کنید.

```
http://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/index.html
```

حال می‌توانیم کد را به نحوی تغییر دهیم تا بر روی **GPU** اجرا شود.

```
    float *d_A, *d_B, *d_C;
    cudaMalloc((float**)&d_A, nBytes);
    cudaMalloc((float**)&d_B, nBytes);
    cudaMalloc((float**)&d_C, nBytes);
```

حال از تابع **cudaMemcpy** برای انتقال اطلاعات از حافظه‌ی **host** به حافظه‌ی **global** کارت گرافیک استفاده می‌کنیم، پارامتر **cudaMemcpyHostToDevice** مشخص کننده ی نوع انتقالی است که قرار است صورت بگیرد.

```
    cudaMemcpy(d_A,h_A,nElem,cudaMemcpyHostToDevice);
    cudaMemcpy(d_B,h_B,nElem,cudaMemcpyHostToDevice);
```

وقتی که داده به حافظه‌ی **global** کارت گرافیک انتقال داده شد، تابع کرنل می‌تواند از طرف **host** برای جمع دو آرایه بر روی **GPU** صدا زده شود. به محض این که کرنل صدا زده شود، سریعا کنترل به دست **host** برمیگردد. در این نقطه، **host** می‌تواند توابع دیگری را اجرا کند و در همین زمان **GPU** نیز پردازش خودش را انجام بدهد. بنابراین **kernel** نسبت به **host** یک پردازش آسنکرون انجام می‌دهد.

وقتی که کرنل کار پردازش بر روی همه‌ی المان‌ها را تمام کرد، نتیجه بر روی حافظه‌ی گلوبال **GPU** و در آرایه‌ی **d_c** ذخیره شده است. حال باید نتیجه‌ی عملیات **GPU** را به آرایه **gpuRef** بر روی **host** برگردانیم و این کار دوباره با استفاده از تابع **cudaMemcpy** انجام میپذیرد.

```
float *gpuRef;
cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyDeviceToHost);
```

فراخوانی کردن **cudaMemcpy** موجب بلاک شدن پردازش **host** می‌شود. نتیجه‌ای که بر روی آرایه‌ی **d_c** بر روی **GPU** قرار داد به **gpuRef** کپی می‌شود، آرگومان **cudaMemcpyDeviceToHost** به این منظور استفاده شده. در نهایت، حافظه‌ای که بر روی **GPU** گرفته شده بود را با استفاده از تابع **cudaFree** آزاد می‌کنیم.

```
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
```

## ۳ تفاوت فضای حافظه host و device

**یکی از رایج‌ترین اشتباهاتی که یک برنامه‌نویس تازه کار در CUDA C انجام میدهد که به صورت اشتباهی بین حافظه‌های dereference کند. برای حافظه‌ای که بر روی GPU اختصاص داده شده، پوینتر ممکن است به درستی بر روی هاست dereference شود. اگر کد شما به طرز اشتباهی عملیات انتساب زیر را انجام بدهد:**

```
gpuRef=d_C;
```

**به جای این که از شکل درست زیر استفاده کند:**

```
cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyDeviceToHost);
```

**برنامه شما ممکن است در runtime با خطا روبه‌رو شده و کرش کند. برای کمک به این مشکل در CUDA 6 یک حافظه‌ی یکپارچه معرفی شده که به شما اجازه می‌دهد که هم بر روی CPU و هم بر روی GPU با استفاده از یک اشاره‌گر تنها دسترسی داشته باشید. در مورد این حافظه یکپارچه در فصل چهارم بیشتر یاد خواهیم گرفت.**

## ۳ چرا در زبان c از pointer to pointer استفاده می‌شود.

ما می‌دانیم که یک اشاره‌گر متغیری است که به یک مکان حافظه اشاره می‌کند و ما از اشاره‌گرها برای ذخیره‌ی آدرس متغیرها استفاده می‌کنیم. بنابراین وقتی که از اشاره‌گر به اشاره‌گر استفاده می‌کنیم. اشاره‌گر اول آدرس اشاره‌گر دوم را در خود ذخیره می‌کند. به همین دلیل آن‌ها را **double pointer** نیز می‌نامیم.

یک دلیل برای استفاده از **pointer to pointer** این است که می‌خواهید مقدار اشاره‌گری که به تابع پاس داده شده را به عنوان آرگومان تابع عوض کنید، بدین منظور نیاز به **pointer to pointer** خواهید داشت.

به بیان ساده‌تر فرض کنید که شما یک پوینتر دارید، برای تغییر دادن آدرس این پوینتر شما به سادگی می‌توانید این اشاره‌گر را برابر با یک اشاره‌گر دیگر قرار دهید، ولی چه می‌شود اگر بخواهیم آدرس این اشاره‌گر را در داخل یک تابع عوض کنیم، مثال زیر را ببینید،

```
int a=2,b=3;
int* pa=&a;
int* pb=&b;
```

در این مثال، ما دو متغیر به نام‌های **a,b** داریم و دو اشاره‌گر به نام **pa,pb** که اشاره‌گر به **a** و **b** هستند. اگر بخواهیم که **pa** مقدار **pb** را بگیرد به سادگی می‌توانیم این کار را با **pa=pb** قرار دادن انجام دهیم، ولی فرض کنید که بخواهیم این کار را درون یک تابع انجام بدهیم.

```
void change_location(int *add1,int* add2)
{
   add1=add2;
}
```

آیا پس از اجرای این تابع به صورت زیر

```
change_location(pa,pb);
```

آدرس **pa** برابر با **pb** خواهد بود؟ پاسخ نه می‌باشد. چون در نهایت **pa,pb** دو متغیر هستند که به صورت **local** مقدارشان عوض می‌شود و تاثیری در بیرون تابع نمی‌گذارند، پس چطور باید این کار را انجام بدهیم؟ پاسخ این است که باید از یک اشاره‌گر دیگر استفاده کنیم. یعنی تابع ما به صورت زیر تغییر پیدا خواهد کرد.

```
void change_location(int **add1,int **add2)
{
   *add1=*add2;
}
```

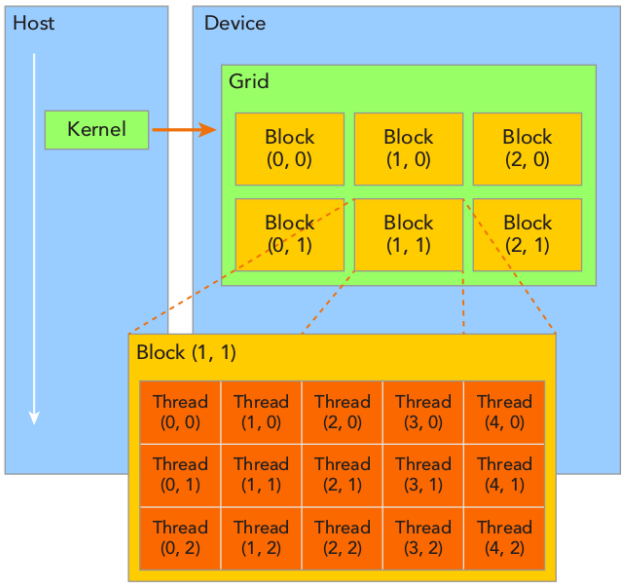
و نحوه‌ی فراخوانی این تابع نیز به صورت زیر خواهد بود.

```
change_location(&pa,&pb);
```

## ۴ مدیریت تردها

وقتی که یک تابع کرنل از سمت **host** اجرا می‌شود، وظیفه‌ی اجرای کرنل بر عهده‌ی **device** گذاشته می‌شود، جایی که تعداد زیادی ترد تولید شده و هر ترد عبارتی که توسط تابع کرنل مشخص شده را اجرا می‌کند. دانستن این که چطور این تردها را می‌توان کنترل کرد یک بخش خیلی مهم در زبان برنامه‌نویسی **CUDA** می‌باشد. **CUDA** در مورد سلسله مراتب این تردها دسترسی‌هایی به ما داده که

بتوانیم تردها را مدیریت کنیم. در واقع ما دو سلسله مراتب تـرد داریم کـه بـه **block ها** و **gridهایی از این block ها** تقسیم می‌شود. شکل زیر را ببینید:




همه‌ی تردهایی که توسط یک کرنل اجرا می‌شوند را مجموعا **grid** می‌نامیم. همه‌ی تردها در درون یـک **grid** ، یک حافظه‌ی **global** را با همدیگر سهیم هسـتند. یـک **grid** از مجمـوعه‌ای از **block** هـا **thread** ساخته شده. یک **block** گروهی از تردها هستند که می‌توانند با همـدیگر **همکاری** کننـد، این همکاری با استفاده از دو مورد زیر انجام می‌شود:

- Block-local synchronization
- Block-local shared memory

تردهایی که از بلاک‌های متفاوت هستند نمی‌توانند با همدیگر همکاری داشته باشند.

تردها از دو مختصات یکتای زیر برای مشخص کردن همدیگر استفاده می‌کنند، یعنی هر ترد را می‌تـوان با مشخص کردن این دو مختصات به صورت یکتا تعریف کرد، و ترد دیگری با این مختصات وجود ندارد:

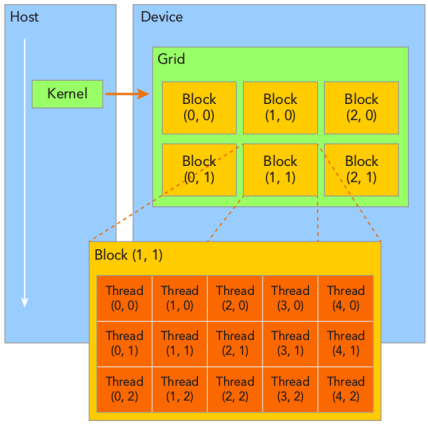
- blockIdx (block index within a grid)
- threadIdx (thread index within a block)

این متغیرها به صورت داخلی ساخته شده‌اند، مقداردهی شده‌اند و می‌توان توسط تابع کرنل به این مقادیر دسترسی داشت.(یعنی چیزی نیست که ما بتوانیم برای یک ترد تغییرش دهیم). وقـتی کـه یـک تابع کرنل اجرا می‌شود، مختصات **blockIdx** و **threadIdx** توسط **CUDA runtime** به هر ترد انتصاب داده می‌شود. بر اساس این مختصات، می‌توانید قسمت‌های مختلف داده را به تردهای مختلف بدهید.

متغیرهای مختصاتی از نوع **uint3** هستند، این **type** توسط **CUDA** و به صورت داخلی ساخته شده و یک بردار می‌باشد، که از **integer** عادی مشتق شده. در ساختار این **type** جدید ما سه عـــدد **integer** داریم، و اولین،دومین و سومین عضو این بردار را می‌توان با فیلدهای **x,y** و **z** به ترتیب به دست آورد.

```
blockIdx.x
blockIdx.y
blockIdx.z
threadIdx.x
threadIdx.y
threadIdx.z
```

**CUDA** در سه بعد **grid** ها و **block** ها را سازماندهی می‌کند. شکل زیر یک مثال از ساختار سلسله مراتبی تردها در یک **grid** دو بعدی است که حاوی **block** های دو بعدی می‌باشد.




ابعاد **grid** و **block** را می‌توان توسط دو متغیر داخلی زیر به دست آورد.

- ➤ blockDim (block dimension, measured in threads)
- ➤ gridDim (grid dimension, measured in blocks)

این متغیرها از نوع **dim3** هستند، که یک بردار از اعداد صحیح هستند که بر اساس **uint3** ساخته شده است تا مشخص کننده‌ی ابعاد باشد. وقتی که یک متغیر از نوع **dim3** تعریف می‌کنید، هر کامپوننتی کـه نامشخص باشد به ۱ مقداردهی می‌شود. هر کامپوننت در متغیر از نوع **dim3** را می‌توان توسط **x,y** و فیلد **z** به دست آورد که در زیر نمایش داده شده است:

```
blockDim.x
blockDim.y
blockDim.z
```

**معمولا، یک grid از آرایه‌های دو بعدی از block ها ساخته شده است، و یک block از یک ساختار سه بعدی تردها تشکیل یافته است.**
**هم grid ها و هم block ها از dim3 استفاده می‌کنند. Field هایی که مورد استفاده قرار نگیرد به ۱ انتساب داده می‌شود و نادیده گرفته می‌شود.**

در برنامه‌ی **CUDA** دو مجموعه متغیر متفاوت وجود دارد: یکی نوع **dim3** می‌باشد که به صورت دستی تعریف می‌شود و یکی **uint3** هست که قبلا تعریف شده. در سمت **host**، می‌توانید ابعاد یک **grid** و **block** را با استفاده از نوع **dim3** به عنوان فراخوانی یک کرنل مشخص کنید. وقتی که کرنل اجرا می‌شود، **CUDA runtime** متغیرهای داخلی، و مقداردهی شده‌ای از **grid** و **block** و **thread** ها را تولید می‌کند که توسط توابع کرنل و به صورت **uint3** قابل دسترسی هستند. متغیرهای دستی **grid** و **block** که به صورت **dim3** هستند، تنها در سمت **host** قابل دیدن هستند، و متغیرهای **built-in**، که از نوع **uint3** هستند تنها برای **device** قابل رویت هستند.

ما این متغیرها را در برنامه‌ی جمع دو آرایه که قبلا به زبان **C** نوشتیم استفاده خواهیم کرد تا بدانید این متغیرها چطور کار می‌کنند. ابتدا، اندازه‌ی داده‌ای که باید در برنامه استفاده شود را مشخص می‌کنیم، به منظور این که توضیح خروجی راحت‌تر باشد، اندازه‌ی داده را کوچک در نظر می‌گیریم.

```
int nElem=6;
```

در ادامه، اندازه‌ی **block size** را تعریف می‌کنیم و **grid size** را بر حسب **block** و اندازه‌ی داده مشخص می‌کنیم. در مورد این مثال، یک **block 1D** تعریف می‌شود که سه ترد دارد و یک **grid 1D** با تعداد بلاک‌هایی که بر اساس اندازه‌ی **data** و اندازه‌ی **block** می‌باشد.

```
dim3 block(3);
dim3 grid((nElem+block.x-1)/block.x);
```

احتمالا متوجه شده‌اید که اندازه‌ی **grid** با استفاده از تعداد بلاک‌ها روند شده است. نگران نباشید، علت این که چرا باید **grid size** را به این شکل محاسبه کنید را در فصل بعد بحث خواهیم کرد. کد زیر در سمت **host** ابعاد **grid** و **block** را چک می‌کند.

```
printf("grid.x %d grid.y %d grid.z %d\n",grid.x, grid.y, grid.z);
printf("block.x %d block.y %d block.z %d\n",block.x, block.y, block.z);
```

در تابع کرنل، هر ترد اندیس خودش، اندیس **block**، ابعاد **block**، و ابعاد **grid** را می‌تواند به صورت زیر چاپ کند.

```
printf("threadIdx:(%d, %d, %d) blockIdx:(%d, %d, %d) blockDim:(%d, %d, %d) "
    "gridDim:(%d, %d, %d)\n", threadIdx.x, threadIdx.y, threadIdx.z,
    blockIdx.x, blockIdx.y, blockIdx.z, blockDim.x, blockDim.y, blockDim.z,
```

```
        gridDim.x,gridDim.y,gridDim.z);
```

اگر کدها را در کنار هم قرار دهیم، کد جدیدمان به صورت زیر خواهد شد:

```
#include <cuda_runtime.h>
#include <stdio.h>
__global__ void checkIndex(void) {
   printf("threadIdx:(%d, %d, %d) blockIdx:(%d, %d, %d) blockDim:(%d, %d, %d) "
         "gridDim:(%d, %d, %d)\n", threadIdx.x, threadIdx.y, threadIdx.z,
         blockIdx.x, blockIdx.y, blockIdx.z, blockDim.x, blockDim.y, blockDim.z,
         gridDim.x,gridDim.y,gridDim.z);
}
int main(int argc, char **argv) {
   // define total data element
   int nElem = 6;
   // define grid and block structure
   dim3 block (3);
   dim3 grid ((nElem+block.x-1)/block.x);
   // check grid and block dimension from host side
   printf("grid.x %d grid.y %d grid.z %d\n",grid.x, grid.y, grid.z);
   printf("block.x %d block.y %d block.z %d\n",block.x, block.y, block.z);
   // check grid and block dimension from device side
   checkIndex <<<grid, block>>> ();
   // reset device before you leave
   cudaDeviceReset();
   return(0);
}
```

حال می‌توانید کد را به صورت زیر اجرا کنید:

```
$ nvcc -arch=sm_20 checkDimension.cu -o check
$ ./check
```

از آنجایی که تابع **printf** تنها بر روی معماری‌های بعد از **Fermi** جواب میدهد، آپشن **-arch=sm_20** را باید اضافه کنیم. به صورت پیش فرض، **nvcc** کد را برای پایین‌ترین معماری تولید می‌کند. نتیجه‌ی این برنامه به صورت زیر خواهد بود. می‌توانید ببینید که هر ترد مختصات خود را دارد، **و همه‌ی تردها دارای ابعاد یکسان block و grid هستند.**

```
grid.x 2 grid.y 1 grid.z 1
block.x 3 block.y 1 block.z 1
threadIdx:(0, 0, 0) blockIdx:(1, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(1, 0, 0) blockIdx:(1, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(2, 0, 0) blockIdx:(1, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(0, 0, 0) blockIdx:(0, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(1, 0, 0) blockIdx:(0, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(2, 0, 0) blockIdx:(0, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
```

## ۴ دسترسی به متغیرهای GRID/BLOCK از سمت HOST و DEVICE

**این خیلی مهمه که بتونیم بین متغیرهای grid,block در سمت host و device تفاوت قایل شویم. به طور مثال، با استفاده از یک متغیر تعریف شده به عنوان block در سمت host، می‌توانیم مختصات را تعریف کرده و به شکل زیر به آن ها دسترسی داشته باشیم.**

**block.x , block.y *and* block.z**

**در سمت دیگر، شما یک سری متغیر از پیش مقداردهی شده، و داخلی دارید که به صورت زیر در دسترس می‌باشد.**

**blockDim.x, blockDim.y *and* blockDim.z**

**به طور خلاصه، شما متغیرها را برای grid و block روی host تعریف کرده، این کار قبل از اجرای یک کرنل صورت می‌پذیرد، و با استفاده از فیلدهای x,y و z به ساختار این بردار در سمت host دسترسی داشته باشیم. وقتی که کرنل اجرا شد، شما می‌توانید از متغیرهای داخلی و از پیش مقداردهی شده درون کرنل استفاده کنید.**

برای یک اندازه‌ی داده خاص، گام‌های عمومی که برای مشخص کردن اندازه‌ی **grid** و **block** وجود دارد عبارت‌است از:

- تصمیم بگیرید که اندازه‌ی **block size** چقدر می‌باشد.
- ابعاد **grid** را بر اساس اندازه‌ی داده و **block size** مشخص کنید.

برای مشخص کردن ابعاد **block**، معمولا باید دو مورد زیر را لحاظ کنید:

- ➤ Performance characteristics of the kernel
- ➤ Limitations on GPU resources

در مورد این موضوع در مثال‌هایی که در ادامه در فصول مختلف خواهیم آورد صحبت خواهیم کرد. کدی که در ادامه خواهیم آورد از یک **grid** یک بعدی و **block** یک بعدی استفاده می‌کند، و **block size** تغییر می‌کند و به همین خاطر **grid size** نیز تغییر می‌کند.

```
#include <cuda_runtime.h>
#include <stdio.h>
```

```
int main(int argc, char **argv) {
    // define total data elements
    int nElem = 1024;
    // define grid and block structure
    dim3 block (1024);
    dim3 grid ((nElem+block.x-1)/block.x);
    printf("grid.x %d block.x %d \n",grid.x, block.x);
    // reset block
    block.x = 512;
    grid.x = (nElem+block.x-1)/block.x;
    printf("grid.x %d block.x %d \n",grid.x, block.x);
    // reset block
    block.x = 256;
    grid.x = (nElem+block.x-1)/block.x;
    printf("grid.x %d block.x %d \n",grid.x, block.x);
    // reset block
    block.x = 128;
    grid.x = (nElem+block.x-1)/block.x;
    printf("grid.x %d block.x %d \n",grid.x, block.x);
    // reset device before you leave
    cudaDeviceReset();
    return(0);
}
```

این برنامه را کامپایل کرده و با استفاده از دستور زیر آن را اجرا می‌کنیم:

```
$ nvcc defineGridBlock.cu -o block
$ ./block
```

خروجی به صورت زیر خواهد بود. از آنجایی که اندازه‌ی داده در این برنامه ثابت بود، وقتی که **block size** تغییر می‌کند، بایستی **grid size** نیز تغییر کند.

```
grid.x 1        block.x 1024
grid.x 2        block.x 512
grid.x 4        block.x 256
grid.x 8        block.x 128
```

**یکی از ویژگی‌هایی که CUDA را از سایر سیستم‌های چند تردی متمایز می‌کند، سلسله مراتب دو سطحی ترد در مدل برنامه‌نویسی آن هست. به این دلیل که ابعاد grid و block در کارآیی کرنلی که اجرا می‌شود، تاثیر دارد، در اختیار گذاشتن این abstraction به برنامه‌نویس، می‌تواند منجر به یک راه برای بهینه کردن شود.**
**چندین محدودیت روی ابعاد grid ها و block ها خواهیم داشت. یکی از بزرگترین محدودیت‌ها روی block size میزان منابع محاسباتی موجود است، مثل رجیسترها، حافظه‌ی اشتراکی، و مواردی از این دست. اطلاعات برخی از این محدودیت‌ها را می‌توان با فرستادن یک درخواست از خود GPU گرفت.**
**Grid ها و block ها در واقع یک روشی برای پیدا کردن دید از سلسله مراتب تردهای تابع کرنل است. در فصل ۳، خواهید دید که این نوع از سازماندهی تردها به شما اجازه میدهد که به طور کارآمدتری یک برنامه‌ی یکسان را بر روی device های متفاوت اجرا کنید، که هر کدام دارای میزان منابع و حافظه متفاوت هستند.**

## ۵ راه‌اندازی یک کرنل CUDA

شما با **syntax** زیر از زبان **C** آشنا هستید:

```
function_name (argument list);
```

یک فراخوانی از کرنل **CUDA** در واقع یک افزونه‌ی مستقیم به زبان **C** هست که فقط تنظیمات کرنلی که قرار است اجرا شود را در داخل **triple-angle-brackets** میگیرد:

```
kernel_name <<<grid, block>>>(argument list);
```

همانطور که در قسمت قبلی توضیح داده شد، مدل برنامه‌نویسی **CUDA** سلسله مراتب ترد را در اختیار شما قرار میدهد، و می‌توانید مشخص کنید که چطور تردها برای اجرا روی **GPU** برنامه‌ریزی شوند. اولین مقداری که به کرنل پاس داده می‌شود ابعاد **grid** بود یعنی تعداد بلاک‌هایی که قرار است راه‌اندازی شود. مقدار دوم ابعاد **block** می‌باشد، یعنی تعداد تردهایی که قرار است روی هر **block** اجرا شود. با مشخص کردن ابعاد **grid** و **block** ، در واقع تنظیمات زیر را انجام داده‌اید:

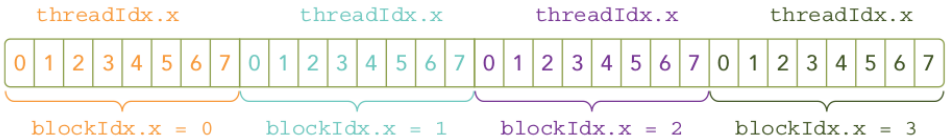
- تعداد کل تردهایی که بر روی یک کرنل اجرا می‌شود.
- طرح تردهایی که می‌خواهید بر روی کرنل استفاده شود.

تردهایی که درون یک بلاک هستند به راحتی می‌توانند با همدیگر ارتباط داشته باشند، و تردهایی که متعلق به بلاک‌های مختلف هستند نمی‌توانند با همدیگر ارتباطی داشته باشند. برای یک مساله‌ی داده شده، شما می‌توانید از **grid** و **block** متفاوتی استفاده کنید. به طور مثال، فرض کنید که شما ۳۲ تا داده

دارید که باید یک سری محاسبه بر روی آن انجام شود. شما می‌توانید این داده را به گروه‌های هشت‌تایی شکسته و هر گروه را به یک بلاک دهید، و چهار بلاک را راه اندازی کنید:

```
kernel_name<<<4, 8>>>(argument list);
```

شکل زیر طرح تردها را برای تنظیمات بالا نشان میدهد.




از آنجایی که داده به صورت خطی در حافظه‌ی **global** ذخیره می‌شود، می‌توانید از متغیرهای داخلی **blockIdx.x** و **threadIdx.x** برای دو منظور زیر استفاده کنید:

- مشخص کردن یک ترد خاص در یک **grid**
- مشخص کردن یک نگاشت بین تردها و المان‌های داده ها.

اگر ۳۲ المان را در یک بلاک قرار دهیم، در این صورت نحوه‌ی فراخوانی کرنل به صورت زیر تغییر خواهد کرد.

```
kernel_name<<<1, 32>>>(argument list);
```

اگر بخواهیم که به هر بلاک فقط یک المان از داده را بدهیم، در این صورت ۳۲ بلاک خواهیم داشت که به صورت زیر فراخوانی می‌شود.

```
kernel_name<<<32, 1>>>(argument list);
```

فراخوانی یک کرنل، یک پروسه آسنکرون نسبت به **host** می‌باشد. وقتی که یک کرنل فراخوانی شد، بلافاصله کنترل دوباره به دست **host** سپرده می‌شود. می‌توانید از دستور زیر استفاده کنید تا **host** را مجبور کنید که منتظر تمام شدن همه‌ی کرنل‌ها بشیند.

```
cudaError_t cudaDeviceSynchronize(void);
```

برخی از **API** های **CODA runtime** یک سنکرون سازی غیر صریح بین **host** و **device** را انجام میدهند. وقتی که از دستور **cudaMemcpy** به منظور کپی کردن داده بین **host** و **device** استفاده می‌کنید، یک سنکرون سازی غیر صریح در سمت **host** اتفاق می‌افتد و برنامه‌ی سمت **host** باید منتظر بماند تا داده به صورت کامل کپی شود.

```
cudaError_t cudaMemcpy(void* dst, const void* src, size_t count, cudaMemcpyKind kind);
```

وقتی این تابع شروع به کپی کردن می‌کند که همه‌ی فراخوانی‌های قبلی از کرنل کامل شده باشند. وقتی که کپی کامل شد، سریعا کنترل به سمت **host** برمیگردد.

**برخلاف فراخوانی یک تابع C، همه‌ی توابع کرنل به صورت آسنکرون آغاز به کـــار می‌کننـــد، و بعـــد از آن کنترل سریعا دست CPU سپرده میشود.**

## ۶ نوشتن کرنل

تابع کرنل تابعی است که در سمت **device** اجرا می‌شود. در تابع کرنل، شما یک محاسبه‌ای را برای اجـــرا بر روی یک ترد مشخص می‌کنید، و دیتایی که برای دسترسی به آن ترد نیاز دارید نیز مشخص می‌کنید. وقتی که کرنل فراخوانی می‌شود، تردهای مختلف **CODA** همان محاسباتی که مشخص کرده بودید را بر به صورت پارالل اجرا می‌کنند. یک کرنل با استفاده از اعلان **__global__** مشخص می‌شود. که در ادامه یک لیست از این اعلان‌ها آورده‌ایم.

```
__global__ void kernel_name(argument list);
```

یک تابع کرنل حتما باید در خروجی **void** داشته باشد.

جدول زیر، یـک خلاصـه‌ای از این توابـع مشـخص‌کننده در برنامه‌نویسـی **C** را نشـان میدهـد. در واقـع **qualifier** مشخص می‌کند که یک تابع بر روی **host** اجرا شود و یا بر روی **device** و یا این که قابلیت این را داشته باشد که از دو سمت نیز فراخوانی شود.

**TABLE 2-2:** Function Type Qualifiers

| QUALIFIERS | EXECUTION | CALLABLE | NOTES |
|---|---|---|---|
| `__global__` | Executed on the device | Callable from the host<br>Callable from the device for devices of compute capability 3 | Must have a `void` return type |
| `__device__` | Executed on the device | Callable from the device only | |
| `__host__` | Executed on the host | Callable from the host only | Can be omitted |

**Qualifiers** های **__device__** و **__host__** می‌توانند با همدیگر استفاده شوند، و آن موقعی اسـت کـه تابع هم برای **host** و هم برای **device** کامپایل شود.

محدودیت‌های زیر بر روی همه‌ی کرنل‌ها اعمال می‌شود:

- ✔ تنها به حافظه‌ی **device** دسترسی دارند.
- ✔ بایستی نوع خروجی **void** باشد.
- ✔ از تعداد آرگومان‌های متغیر پشتیبانی نمی‌کنند.
- ✔ از متغیرهای **static** پشتیبانی نمی‌کنند.
- ✔ از اشاره‌گر به تابع پشتیبانی نمی‌کنند.
- ✔ یک رفتار آسنکرون دارند.

به عنوان نمونه، یک مثال ساده از جمع کردن دو بردار **A,B** با سایز **N** را در نظر بگیرید. برنامه‌ای که برای جمع این دو بردار در زبان **C** نوشته شده بود به صورت زیر بود:

```
void sumArraysOnHost(float *A, float *B, float *C, const int N) {
    for (int i = 0; i < N; i++)
        C[i] = A[i] + B[i];
}
```

این یک برنامه‌ی ترتیبی است که که **N** بار یک کاری را انجام میدهد. با خلاص شدن از حلقه به منظور نوشتن یک کرنل برنامه به صورت زیر درمیآید.

```
__global__ void sumArraysOnGPU(float *A, float *B, float *C) {
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}
```

تفاوت بین تابع در زبان **C** و تابع کرنل چیست؟ واضح است تابع کرنل حلقه ندارد، مختصات تردها به منظور اندیس آرایه استفاده شده، و هیچ رفرنسی به **N** وجود ندارد، در واقع این کار به صورت غیر صریح و مشخص کردن **N** ترد برای اجرا قبلا مشخص شده است.

فرض کنید که یک بردار به طول ۳۲ داریم، شما می‌توانید یک کرنل با ۳۲ ترد به صورت زیر بر روی آن اجرا کنید.

```
sumArraysOnGPU<<<1,32>>>(float *A, float *B, float *C);
```

## ۷ صحت سنجی کرنل

بعد از این که کرنل را نوشتید، از کجا می‌توانیم بفهمیم که به درستی کار می‌کند؟ در این صورت نیاز به یک تابع **host** داریم که نتایج کرنل را تصدیق کند.

```
void checkResult(float *hostRef, float *gpuRef, const int N) {
    double epsilon = 1.0E-8;
    int match = 1;
    for (int i = 0; i < N; i++) {
        if (abs(hostRef[i] - gpuRef[i]) > epsilon) {
            match = 0;
            printf("Arrays do not match!\n");
            printf("host %5.2f gpu %5.2f at current %d\n",
                hostRef[i], gpuRef[i], i);
            break;
        }
    }
    if (match) printf("Arrays match.\n\n");
    return;
}
```

۷ تصدیق کد کرنل

**در کنار، ابزارهای دیباگی که وجود دارد، دو روش ابتدایی برای تصدیق کد کرنل وجود دارد.**
**اول این که، می‌توانید از printf برای نسل Fermi و نسل‌های بعدتر استفاده کنید.**
**دوم، می‌تونید کانفیگ اجرای کرنل را بر روی <<<۱,۱>>> قرار دهید، یعنی کرنل را مجبور کنید که تنها بر روی یک بلاک و بر روی ترد کار کند. این یک پیاده‌سازی ترتیبی خواهد بود. این برای دیباگ کردن و صحت سنجی نتایج درست بسیار مفید خواهد بود. همچنین، این به شما کمک می‌کند تا راحت‌تر بتوانید مشخص کنید که نتایج به صورت بیت به بیت با هم یکسان هستند، مخصوصا اگر با مسایلی روبه‌رو هستید که حالت ترتیبی دارند.**

## ۸ رسیدگی به خطاها

از آنجایی که بیشتر فراخوانی‌ها **CODA** به صورت آسنکرون هستند، مشخص کردن این که کدام روتین منجر به خطا شده است مشکل است. مشخص کردن یک ماکرو برای رسیدگی به خطا که به عنوان پوششی برای **CODA API** باشد منجر به سادگی چک کردن خطاها می‌شود.

```
#define CHECK(call)
{
    const cudaError_t error = call;
    if (error != cudaSuccess)
    {
        printf("Error: %s:%d, ", __FILE__, __LINE__);
        printf("code:%d, reason: %s\n", error, cudaGetErrorString(error));
        exit(1);
    }
}
```

به طور مثال، می‌توانید از این ماکرو برای چک کردن کد زیر استفاده کنید.

```
CHECK(cudaMemcpy(d_C, gpuRef, nBytes, cudaMemcpyHostToDevice));
```

اگر کپی کردن حافظه و یا عملیاتی آسنکرون قبلی منجر به یک خطا شود، ماکرو گزارش خطا را خواهد داد، و یک پیام که توسط انسان قابل خواندن است را گزارش می‌کند، و برنامه را متوقف می‌کند. همچنین می‌توانید از این برنامه اجرای کرنل‌ها نیز به صورت زیر استفاده کنید:

```
    kernel_function<<<grid, block>>>(argument list);
    CHECK(cudaDeviceSynchronize());
```

در اینجا، **CHECK(cudaDeviceSynchronize())** ترد **host** را متوقف می‌کند تا همه‌ی **task** های قبلی اجرا شوند، و اطمینان حاصل شود که هیچ خطایی در اجرای کرنل رخ نداده است. این تکنیک تنها باید در مرحله دیباگ اجرا شود، چرا که اضافه کردن این **check point** بعد از کرنل منجر به متوقف شدن ترد **host** می‌شود و به عنوان یک سد عمل خواهد کرد.

## ۹ کامپایل و اجرا

حال کل کدها را با هم در یک فایل به نام **sumArraysOnGPU_small_case.cu** جمع کرده، این کد به صورت زیر خواهد شد:

```
#include <cuda_runtime.h>
#include <stdio.h>
#define CHECK(call){gpu_error_checker((call),__FILE__,__LINE__);}
inline void gpu_error_checker(cudaError_t error,const char*file, int line)
{
    if (error != cudaSuccess)
    {
        printf("Error: %s:%d, ", __FILE__, __LINE__);
```

```
        printf("code:%d, reason: %s\n", error, cudaGetErrorString(error));
    }
}
void checkResult(float *hostRef, float *gpuRef, const int N) {
    double epsilon = 1.0E-8;
    bool match = 1;
    for (int i=0; i<N; i++) {
        if (abs(hostRef[i] - gpuRef[i]) > epsilon) {
            match = 0;
            printf("Arrays do not match!\n");
            printf("host %5.2f gpu %5.2f at current %d\n",hostRef[i],gpuRef[i],i);
            break;
        }
    }
    if (match) printf("Arrays match.\n\n");
}
void initialData(float *ip,int size) {
    // generate different seed for random number
    time_t t;
    srand((unsigned) time(&t));
    for (int i=0; i<size; i++) {
        ip[i] = (float)( rand() & 0xFF )/10.0f;
    }
}
void sumArraysOnHost(float *A, float *B, float *C, const int N) {
    for (int idx=0; idx<N; idx++)
        C[idx] = A[idx] + B[idx];
}
__global__ void sumArraysOnGPU(float *A, float *B, float *C) {
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}
int main(int argc, char **argv) {
    printf("%s Starting...\n", argv[0]);
    // set up device
    int dev = 0;
    cudaSetDevice(dev);
    // set up data size of vectors
    int nElem = 32;
```

```
printf("Vector size %d\n", nElem);
// malloc host memory
size_t nBytes = nElem * sizeof(float);
float *h_A, *h_B, *hostRef, *gpuRef;
h_A= (float *)malloc(nBytes);
h_B= (float *)malloc(nBytes);
hostRef = (float *)malloc(nBytes);
gpuRef = (float *)malloc(nBytes);
// initialize data at host side
initialData(h_A, nElem);
initialData(h_B, nElem);
memset(hostRef, 0, nBytes);
memset(gpuRef, 0, nBytes);
// malloc device global memory
float *d_A, *d_B, *d_C;
cudaMalloc((float**)&d_A, nBytes);
cudaMalloc((float**)&d_B, nBytes);
cudaMalloc((float**)&d_C, nBytes);
// transfer data from host to device
cudaMemcpy(d_A, h_A, nBytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, nBytes, cudaMemcpyHostToDevice);
// invoke kernel at host side
dim3 block (nElem);
dim3 grid (nElem/block.x);
sumArraysOnGPU<<< grid, block >>>(d_A, d_B, d_C);
printf("Execution configuration <<<%d, %d>>>\n",grid.x,block.x);
// copy kernel result back to host side
cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyDeviceToHost);
// add vector at host side for result checks
sumArraysOnHost(h_A, h_B, hostRef, nElem);
// check device results
checkResult(hostRef, gpuRef, nElem);
// free device global memory
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
// free host memory
free(h_A);
free(h_B);
```

```
    free(hostRef);
    free(gpuRef);
    return(0);
}
```

تنظیمات اجرا در این مورد بر روی یک بلاک تنظیم شده است، که شامل ۳۲ المان می‌باشد:

```
dim3 block (nElem);
dim3 grid (nElem/block.x);
```

این کد را می‌توانید با استفاده از کامندهای زیر کامپایل و اجرا کنید:

```
$ nvcc sumArraysOnGPU-small-case.cu -o addvector
$ ./addvector
```

نتیجه‌ای که سیستم خروجی می‌دهد یک چیزی به مانند زیر است.

```
cuda_c_code Starting...
Vector size 32
Execution configuration <<<1, 32>>>
Arrays match.
```

اگر تنظیمات اجرا را بر روی ۳۲ بلاک قرار دهید، در این صورت هر بلاک تنها یک المان را میگیرد:

```
dim3 block (1);
dim3 grid (nElem);
```

در این صورت نیاز خواهید داشت که در تابع کرنل **sumArraysOnGPU** خط زیر را

```
int i = threadIdx.x;
```

با خط زیر جابه‌جا کنید:

```
int i = blockIdx.x;
```

در حالت کلی می‌توانید اندیس یکتایی را برای دسترسی به دیتا به صورت زیر تعریف کنید، باید توجه داشته باشید که این دستور بر اساس **grid** و **block** یک بعدی نوشته شده است:

```
__global__ void sumArraysOnGPU(float *A, float *B, float *C)
{
    int i = blockIdx.x*blockDim.x+threadIdx.x;
    C[i] = A[i] + B[i];
}
```

در واقع شما باید این تغییر رو لحاظ کنید تا مطمعن شوید که در حالت کلی جواب درستی به شما داده می‌شود.

## ۱۰ گرفتن تایم کرنل

این که بدانیم چه میزان زمان لازم است تا کرنل اجرا شود برای ما سودمند می‌باشد، همچنین موقع ارزیابی کارآمدی کرنل این یک کار ضروری به حساب می‌آید. چندین روش برای گرفتن زمان اجرای کرنل وجود دارد. ساده‌ترین راه استفاده از تایمر **CPU** و یا تایمز **GPU** می‌باشد، که از سمت **host** اندازه گیری می‌شود. در این قسمت ما یک تایمر **CPU** خواهیم ساخت، همچنین یاد خواهیم گرفت که چطور از **NVIDIA profiler** استفاده کنیم.

فصل شش به شما آموزش خواهد داد که چطور از روتین‌های گرفتن زمان **CODA** استفاده کنید.

## ۱۱ گرفتن زمان اجرا با استفاده از تایمر CPU

تایمر **CPU** را می‌توان با استفاده از فراخوانی از سیستم **gettimeofday** گرفت، که تعداد ثانیه‌هایی که از مبدا تاریخ سیستم زده شده را برمیگرداند. برای این منظور لازم است که هدر فایل **sys/time.h** را نیز در برنامه بیاورید.

```
double cpuSecond(){
    struct timeval tp;
    gettimeofday(&tp,NULL);
    return ((double)tp.tv_sec+(double)tp.tv_usec*1.e-6);
}
```

حال می‌توانیم با استفاده از این تابع زمان اجرای کرنل را به صورت زیر به‌دست‌بیاوریم.

```
    double iStart=cpuSecond();
    sumArraysOnGPU<<< grid, block >>>(d_A, d_B, d_C);
    cudaDeviceSynchronize();
    double iElaps=cpuSecond()-iStart;
    printf("TIME ELAPSED=%f \n",iElaps);
```

که خروجی روی سیستم من به صورت زیر خواهد بود.

```
TIME ELAPSED=0.000023
```

از آنجایی که فراخوانی یک کرنل نسبت به **host** یک رفتار آسنکرون دارد، ما مجبوریم که از تابع سنکرون کردن یعنی تابع **cudaDeviceSynchronize** استفاده کنیم تا **host** منتظر اجرای همه‌ی تردهای **GPU** باشد. متغیر **iElaps** میزان زمان سپری شده را به ثانیه نمایش میدهد.

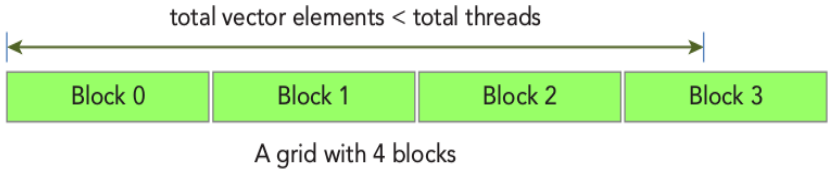
حال برنامه را با یک بردار ۱۶ میلیونی تست می‌کنیم

```
int nElem = 1<<24;
```

برای این منظور بایستی کرنل را به منظور مقیاس‌پذیری **GPU** با استفاده از اندیس **block** و **thread** تصحیح کنیم، و یک شرط نیز برای این اندیس‌ها قرار دهیم تا از مرز رد نشود (**i<N**):

```
__global__ void sumArraysOnGPU(float *A, float *B, float *C,const int N) {
    int i = blockIdx.x*blockDim.x+threadIdx.x;
    if(i<N) C[i] = A[i] + B[i];
}
```

با این تغییرات می‌توانیم کرنل را با تنظیمات مختلفی اجرا کنیم. حال ممکن است حالتی پیش بیاید که تعداد کل تردها بیشتر از تعداد کل بردار باشد، در این حالت باید کرنل را از دسترسی به حافظه‌ی غیر مجاز منع کنید. که در شکل زیر نمایش داده شده است:




در این صورت برنامه به صورت زیر در خواهد آمد.

```
int main(int argc, char **argv) {
    printf("%s Starting...\n", argv[0]);
    // set up device
    int dev = 0;
    cudaSetDevice(dev);
    // set up data size of vectors
    int nElem = 1<<24;
    printf("Vector size %d\n", nElem);
    // malloc host memory
    size_t nBytes = nElem * sizeof(float);
    float *h_A, *h_B, *hostRef, *gpuRef;
    h_A= (float *)malloc(nBytes);
    h_B= (float *)malloc(nBytes);
    hostRef = (float *)malloc(nBytes);
    gpuRef = (float *)malloc(nBytes);
    // initialize data at host side
    initialData(h_A, nElem);
```

```
initialData(h_B, nElem);
memset(hostRef, 0, nBytes);
memset(gpuRef, 0, nBytes);
// malloc device global memory
float *d_A, *d_B, *d_C;
cudaMalloc((float**)&d_A, nBytes);
cudaMalloc((float**)&d_B, nBytes);
cudaMalloc((float**)&d_C, nBytes);
// transfer data from host to device
cudaMemcpy(d_A, h_A, nBytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, nBytes, cudaMemcpyHostToDevice);
// invoke kernel at host side
int iLen=1024;
dim3 block (iLen);
dim3 grid ((nElem+block.x-1)/block.x);
double iStart=cpuSecond();
sumArraysOnGPU<<< grid, block >>>(d_A, d_B, d_C,nElem);
cudaDeviceSynchronize();
double iElaps=cpuSecond()-iStart;
printf("TIME ELAPSED ON GPU=%f \n",iElaps);
printf("Execution configuration <<<%d, %d>>>\n",grid.x,block.x);
// copy kernel result back to host side
cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyDeviceToHost);
// add vector at host side for result checks
iStart=cpuSecond();
sumArraysOnHost(h_A, h_B, hostRef, nElem);
iElaps=cpuSecond()-iStart;
printf("TIME ELAPSED ON CPU=%f \n",iElaps);
// check device results
checkResult(hostRef, gpuRef, nElem);
// free device global memory
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
// free host memory
free(h_A);
free(h_B);
free(hostRef);
free(gpuRef);
```

```
    return(0);
}
```

پس از کامپایل و مشاهده‌ی برنامه می‌توانیم تفاوت پرفرمنس را در **GPU** و **CPU** را ببینیم.

```
Vector size 16777216
TIME ELAPSED ON GPU=0.000660
Execution configuration <<<16384, 1024>>>
TIME ELAPSED ON CPU=0.020561
Arrays match.
```